



Université Cadi Ayyad

Faculté Polydisciplinaire de Safi

Master en Science des Données et Analytique

Modèle de Prédiction de l'Attrition Client Basé sur des Réseaux de Neurones Hybrides

Réalisé par :

Homada Sarra

Belaalia Salma

Encadré par :

Dr. Mounir Ilhame

Année Universitaire : 2025 – 2026

Résumé

La prédiction de l'attrition client constitue un enjeu important pour les entreprises souhaitant améliorer la fidélisation de leurs clients et limiter les pertes financières. Les approches de Machine Learning et de Deep Learning sont largement utilisées pour analyser les comportements clients et identifier les facteurs liés au départ des utilisateurs.

Dans ce projet, nous reproduisons et évaluons un modèle hybride de réseaux de neurones nommé **CCP-Net** (Customer Churn Prediction Network), initialement proposé par Liu et al. [1], dédié à la prédiction de l'attrition client. Ce modèle combine trois techniques principales : **Multi-Head Self-Attention**, **BiLSTM** et **CNN**. Le mécanisme d'attention permet de capturer les dépendances globales entre les données clients, tandis que le modèle BiLSTM prend en compte les relations temporelles. En parallèle, le réseau CNN est utilisé pour extraire les caractéristiques locales et les comportements à court terme.

Afin de traiter le problème du déséquilibre des classes dans les jeux de données, la technique de suréchantillonnage **ADASYN** [4] est appliquée lors de l'étape de prétraitement des données.

Le modèle est étudié sur plusieurs ensembles de données issus de différents domaines, notamment les télécommunications, les banques, les assurances et les services d'actualités. Cette étude met ainsi en évidence l'utilisation des architectures hybrides de Deep Learning dans les systèmes modernes de prédiction de l'attrition client.

Table des figures

3.1	Architecture globale du modèle CCP-Net	14
3.2	Structure du modèle Multi-Head Self-Attention amélioré	15
3.3	Structure du modèle LSTM	17
3.4	Structure du modèle BiLSTM	18
3.5	Structure du modèle CNN	19
4.1	Étapes de projet	21
4.2	Plateforme Kaggle utilisée comme source des datasets	22
4.3	Langage de programmation Python	24
4.4	Éditeur de code Visual Studio Code	25
5.1	Heatmap des corrélations entre les principales variables du dataset Telecom.	32
5.2	Heatmap des corrélations entre les principales variables du dataset Bank.	33
5.3	Heatmap des corrélations entre les principales variables du dataset Insurance.	34
5.4	Heatmap des corrélations entre les principales variables du dataset News.	35
5.5	Code Python d'application de la méthode ADASYN	38
5.6	Distribution des classes après ADASYN pour les datasets Telecom et Bank	40
5.7	Distribution des classes après ADASYN pour les datasets Insurance et News	40
6.1	Code MultiHead Self Attention	43
6.2	Code BiLSTM	43
6.3	Code CCP-Net complet	44
6.4	Phase d'entraînement	47
6.5	Phase de validation et Early Stopping	47
7.1	Courbes de perte et d'accuracy - dataset Telecom	51
7.2	Courbes de perte et d'accuracy - dataset Bank	51
7.3	Courbes de perte et d'accuracy - dataset Insurance	52
7.4	Courbes de perte et d'accuracy - dataset News	52
7.5	Matrice de confusion des datasets Telecom et Bank	53
7.6	Matrice de confusion des datasets Insurance et News	53

7.7	Comparaison des Accuracy de l'étude d'ablation	55
7.8	Comparaison des F1-Scores de l'étude d'ablation	55

Liste des tableaux

2.1	Comparaison des approches de prédiction du churn	11
4.1	Caractéristiques des datasets utilisés	23
5.1	Résumé du nettoyage du dataset Télécom	27
5.2	Résumé du nettoyage du dataset Bank	28
5.3	Résumé de la structure et distribution du dataset Insurance	29
5.4	Résumé du nettoyage du dataset News	29
5.5	Résumé de l'encodage des variables catégorielles	30
5.6	Résultats de la normalisation avec StandardScaler	36
5.7	Résultats de l'équilibrage des classes avec ADASYN	39
6.1	Hyperparamètres utilisés durant l'entraînement	46
7.1	Résultats finaux du modèle CCP-Net sur les différents datasets	49
7.2	Étude d'ablation du modèle CCP-Net sur le dataset Telecom	54

Table des matières

Résumé	1
Liste des Figures	2
Liste des Tableaux	4
1 Introduction	7
1.1 Contexte	7
1.2 Problématique	8
1.3 Objectifs	8
2 État de l'Art	9
2.1 Méthodes de Machine Learning	9
2.2 Méthodes de Deep Learning	10
2.3 Comparaison et limites des approches	11
2.4 Approches hybrides	12
3 Méthodologie	13
3.1 Architecture globale du modèle	13
3.2 Multi-Head Self-Attention	14
3.3 BiLSTM	16
3.4 CNN	19
4 Prérequis et configuration initiale	21
4.1 Acquisition des données	22
4.2 Environnement de développement	23
4.2.1 Langage de développement	23
4.2.2 Outil de développement	25
5 Prétraitement des données	26
5.1 Nettoyage des données	26

5.1.1	Dataset Télécom	27
5.1.2	Dataset Bancaire	28
5.1.3	Dataset Assurance	28
5.1.4	Dataset News	29
5.2	Encodage des variables catégorielles	30
5.3	Analyse des corrélations	30
5.3.1	Dataset Telecom	31
5.3.2	Dataset Bank	32
5.3.3	Dataset Insurance	33
5.3.4	Dataset News	34
5.4	Normalisation des données	35
5.4.1	Équilibrage des données avec ADASYN	37
6	Construction et entraînement du modèle CCP-Net	41
6.1	Architecture du modèle CCP-Net	42
6.2	Entraînement du modèle	45
7	Résultats et Analyse	48
7.1	Métriques d'évaluation	48
7.2	Résultats expérimentaux	49
7.3	Courbes d'apprentissage	50
7.4	Matrices de confusion	53
7.4.1	Matrices de confusion des datasets	53
7.5	Étude d'ablation	54
7.5.1	Résultats de l'étude d'ablation	54
	Conclusion	56
	Bibliographie	56

Chapitre 1

Introduction

Avec l'évolution rapide des technologies numériques et l'intensification de la concurrence entre les entreprises, la fidélisation des clients est devenue un facteur essentiel pour assurer la stabilité et la croissance des organisations. L'attrition client, appelée également *Customer Churn*, désigne le phénomène par lequel un client cesse d'utiliser les services ou les produits d'une entreprise. La prédiction de l'attrition client permet aux entreprises d'identifier les clients susceptibles de quitter leurs services afin de mettre en place des stratégies de rétention adaptées. Grâce aux avancées du Machine Learning et du Deep Learning, il est désormais possible d'analyser de grandes quantités de données clients afin d'anticiper les comportements futurs [5, 6].

Cependant, les modèles traditionnels présentent certaines limites dans l'extraction des relations complexes et des dépendances temporelles présentes dans les données séquentielles. Pour résoudre ces problèmes, ce projet propose une architecture hybride basée sur plusieurs réseaux de neurones profonds afin d'améliorer la précision de la prédiction, en s'appuyant sur les travaux récents de Liu et al. [1] qui ont démontré l'efficacité d'une telle combinaison sur des données issues de plusieurs secteurs d'activité.

1.1 Contexte

Dans les secteurs tels que les télécommunications, les banques, les assurances et les plateformes numériques, la perte de clients peut entraîner des conséquences financières importantes. Les entreprises investissent donc dans des systèmes intelligents capables d'analyser les comportements des utilisateurs afin de réduire le taux d'attrition. Les systèmes CRM (*Customer Relationship Management*) permettent de collecter des informations détaillées sur les clients, comme les historiques de transactions, les habitudes d'utilisation et les interactions avec les services. Ces données constituent une source précieuse pour

construire des modèles prédictifs performants [1]. De nombreux travaux récents confirment l'intérêt croissant porté à ce sujet, en particulier dans le secteur des télécommunications où le coût d'acquisition d'un nouveau client est généralement bien supérieur à celui de la fidélisation d'un client existant [7].

1.2 Problématique

Les approches classiques de Machine Learning telles que la régression logistique, les arbres de décision ou Random Forest présentent des performances limitées lorsqu'il s'agit de traiter des relations non linéaires complexes et des dépendances temporelles [6].

De plus, les jeux de données de churn souffrent généralement d'un fort déséquilibre entre les clients actifs et les clients ayant quitté le service. Cette situation peut entraîner un biais du modèle vers la classe majoritaire et réduire la qualité des prédictions ; c'est pourquoi des techniques de rééchantillonnage adaptatif telles qu'ADASYN sont couramment mobilisées pour corriger ce déséquilibre [4].

Le défi principal du projet consiste donc à développer un modèle hybride capable de :

- Extraire efficacement les caractéristiques globales et locales ;
- Gérer les dépendances temporelles ;
- Traiter le problème du déséquilibre des classes ;
- Améliorer la robustesse et la précision des prédictions.

1.3 Objectifs

Les principaux objectifs de ce projet sont les suivants :

- Étudier les techniques de Machine Learning et de Deep Learning appliquées à la prédiction du churn ;
- Concevoir une architecture hybride basée sur Multi-Head Self-Attention, BiLSTM et CNN ;
- Utiliser la méthode ADASYN pour équilibrer les données ;
- Évaluer les performances du modèle sur différents jeux de données ;
- Comparer les résultats obtenus avec d'autres approches existantes.

Chapitre 2

État de l'Art

La prédiction de l'attrition client (*Customer Churn*) constitue un domaine de recherche important en apprentissage automatique et en intelligence artificielle. Les approches proposées dans la littérature peuvent être regroupées en quatre catégories principales : les méthodes de Machine Learning, les méthodes de Deep Learning, la comparaison et les limites des approches, et enfin les approches hybrides.

2.1 Méthodes de Machine Learning

Les méthodes de Machine Learning représentent les premières approches utilisées pour la prédiction du churn. Elles reposent sur des données structurées et nécessitent généralement une étape de prétraitement et de sélection manuelle des caractéristiques [5].

Parmi les algorithmes les plus utilisés, on retrouve :

- Support Vector Machine (SVM)
- Régression Logistique (LR)
- K-Nearest Neighbors (KNN)
- Decision Tree (DT)
- Random Forest (RF)
- XGBoost

Ces modèles apprennent une fonction de décision :

$$f(x) \rightarrow \{0, 1\}$$

où 1 représente un client susceptible de quitter le service.

Lalwani et al. [6] ont notamment comparé plusieurs de ces algorithmes après une étape de sélection de caractéristiques par recherche gravitationnelle, montrant que les méthodes d'ensemble telles qu'AdaBoost et XGBoost obtiennent généralement les meilleures performances. Plus récemment, Sikri et al. [7] ont proposé une technique de rééquilibrage basée

sur des ratios, combinée à plusieurs classifieurs indépendants, pour améliorer la rétention client dans le secteur des télécommunications.

Les méthodes de Machine Learning :

- nécessitent un feature engineering manuel ;
- fonctionnent bien sur des données structurées ;
- sont faciles à interpréter ;
- sont peu adaptées aux données séquentielles.

2.2 Méthodes de Deep Learning

Les méthodes de Deep Learning permettent d'apprendre automatiquement des représentations complexes et hiérarchiques à partir des données sans intervention humaine importante.

Les architectures les plus utilisées pour la prédiction du churn sont :

- Artificial Neural Networks (ANN)
- Convolutional Neural Networks (CNN)
- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Bidirectional LSTM (BiLSTM)
- Transformers [3]

Les modèles de Deep Learning possèdent plusieurs caractéristiques fondamentales qui les rendent adaptés aux tâches complexes comme la prédiction du churn :

- **Apprentissage automatique des features** : les modèles extraient automatiquement les caractéristiques pertinentes à partir des données brutes sans intervention humaine ;
- **Représentations hiérarchiques** : les couches successives permettent d'apprendre des niveaux d'abstraction de plus en plus élevés ;
- **Capacité à modéliser la non-linéarité** : les fonctions d'activation non linéaires permettent de capturer des relations complexes entre les variables ;
- **Traitement des données séquentielles** : les architectures comme LSTM et BiLSTM permettent de modéliser les dépendances temporelles dans les comportements clients ;
- **Extraction de caractéristiques locales et globales** : CNN capture les patterns locaux tandis que les mécanismes d'attention, introduits par le Transformer [3], capturent les dépendances globales ;
- **Adaptabilité aux grands volumes de données** : ces modèles améliorent leurs performances avec l'augmentation de la quantité de données disponibles.

2.3 Comparaison et limites des approches

Le tableau suivant présente une comparaison structurée des principales approches utilisées dans la prédiction du churn en fonction de leur capacité à traiter différents types de données et leur efficacité dans ce contexte.

Approche	Feature Engineering	Données séquentielles	Non-linéarité	Interprétabilité	Limites dans le churn prediction
SVM / LR / KNN	Forte	Faible	Moyenne	Élevée	Sensibles aux données bruitées et incapables de modéliser des comportements clients complexes
Random Forest / XGBoost	Moyenne	Faible	Bonne	Moyenne	Performants mais dépendants des features et peu adaptés aux dépendances temporelles
ANN	Faible	Moyenne	Bonne	Faible	Ne capture pas efficacement les relations temporelles des comportements clients
CNN	Faible	Moyenne	Bonne	Faible	Limité dans la modélisation des dépendances longues dans les historiques clients
LSTM / BiLSTM	Faible	Forte	Bonne	Faible	Difficulté à capturer les dépendances globales et interactions complexes

TABLE 2.1 – Comparaison des approches de prédiction du churn

Malgré leurs performances respectives, les approches de Machine Learning et de Deep Learning présentent plusieurs limites dans le contexte de la prédiction du churn :

- incapacité des modèles classiques à capturer les interactions complexes entre les variables comportementales ;
- difficulté des modèles séquentiels à modéliser simultanément les dépendances locales et globales ;
- dépendance importante à la qualité et à la représentation des données ;
- sensibilité au déséquilibre des classes, très fréquent dans les datasets de churn ;
- absence d’une architecture unique capable de combiner efficacement les informations locales, globales et temporelles.

Ainsi, ces limitations motivent le développement d’architectures hybrides capables d’exploiter simultanément plusieurs types de représentations.

2.4 Approches hybrides

Afin de surmonter ces limitations, plusieurs travaux récents proposent des architectures hybrides combinant plusieurs réseaux de neurones.

Ces modèles visent à exploiter les forces complémentaires de chaque architecture :

- CNN-LSTM : extraction locale + dépendances temporelles, comme dans le modèle proposé par Zhou et al. [8] pour la prédiction du churn sur des plateformes de streaming musical ;
- BiLSTM-CNN : amélioration de la représentation bidirectionnelle, à l'image du modèle composite de Khattak et al. [2] ;
- LSTM-Attention : focalisation sur les informations importantes ;
- Hybrid Attention Models : combinaison des dépendances globales et locales, comme le modèle HNNSAE fondé sur le Multi-Head Self-Attention proposé par Wu [9].

Ces travaux font par ailleurs suite aux premières tentatives de combinaison de réseaux de neurones pour le churn, telles que le modèle hybride ANN+SOM proposé par Tsai et Lu [10], qui a montré l'intérêt d'associer plusieurs architectures plutôt que de s'appuyer sur un modèle unique.

Cependant, malgré ces améliorations, certaines limites persistent dans la littérature, notamment la difficulté à intégrer simultanément les dépendances globales, locales et temporelles.

C'est dans ce contexte que le modèle proposé dans ce projet reprend l'architecture hybride avancée décrite par Liu et al. [1], combinant [Multi-Head Self-Attention](#), [BiLSTM](#) et [CNN](#).

Cette combinaison permet d'exploiter simultanément les caractéristiques locales, les dépendances temporelles ainsi que les relations globales dans les données clients, afin d'améliorer la performance globale de la prédiction du churn et de mieux modéliser la complexité des comportements des utilisateurs.

Chapitre 3

Méthodologie

Dans ce travail, nous reproduisons et évaluons un modèle hybride de prédiction de l'attrition client nommé **CCP-Net**, dont l'architecture a été initialement proposée par Liu et al. [1]. Ce modèle repose sur la combinaison de trois architectures de deep learning complémentaires afin de mieux capturer la complexité des comportements clients.

L'idée principale est d'exploiter simultanément :

- les dépendances globales entre les caractéristiques clients,
- les dépendances temporelles dans les séquences comportementales,
- les motifs locaux présents dans les données.

3.1 Architecture globale du modèle

Le modèle CCP-Net est composé de trois modules principaux organisés en cascade :

- **Multi-Head Self-Attention** pour l'extraction des dépendances globales ;
- **BiLSTM** pour la modélisation des dépendances temporelles ;
- **CNN** pour l'extraction des caractéristiques locales.

Cette architecture permet une représentation hiérarchique des données clients, améliorant ainsi la capacité de prédiction du modèle.

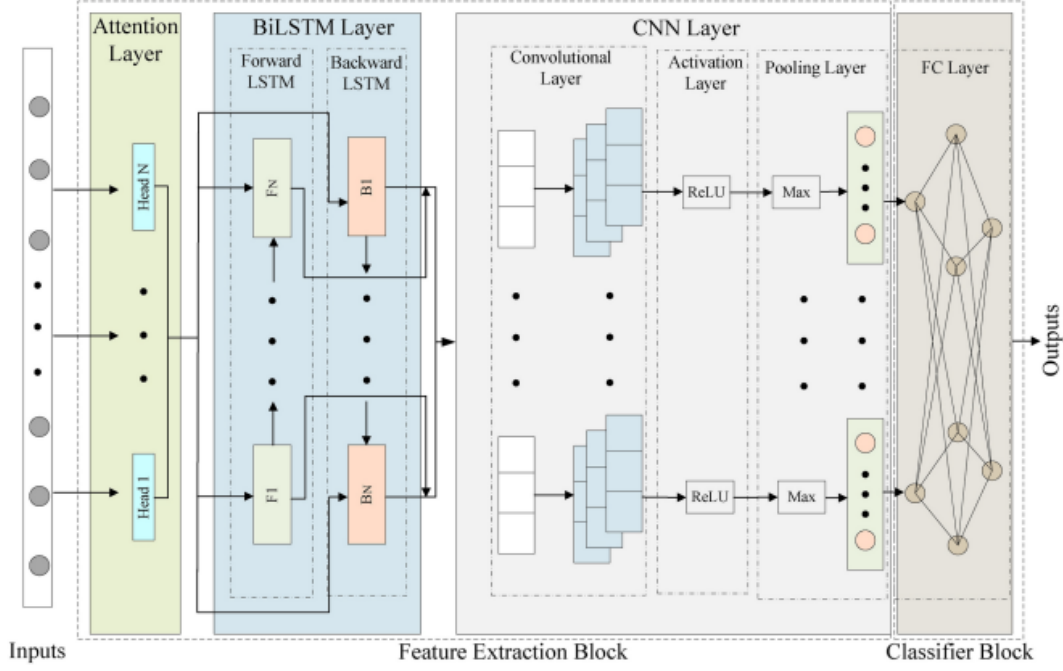


FIGURE 3.1 – Architecture globale du modèle CCP-Net

3.2 Multi-Head Self-Attention

Depuis l'introduction du modèle Transformer [3], le mécanisme Multi-Head Self-Attention est devenu l'une des techniques les plus utilisées pour l'extraction des dépendances globales dans les données séquentielles. Ce mécanisme permet au modèle d'analyser simultanément plusieurs représentations de la séquence d'entrée grâce à plusieurs têtes d'attention fonctionnant en parallèle. Chaque tête apprend des relations différentes entre les variables, ce qui améliore considérablement la qualité des représentations générées.

Dans le contexte de la prédiction du churn client, le module Multi-Head Self-Attention joue un rôle important dans l'identification des comportements et des interactions cachées entre les caractéristiques des clients. En exploitant plusieurs sous-espaces d'apprentissage, le modèle peut mieux capturer les motifs complexes présents dans les données transactionnelles et comportementales, ce qui contribue à améliorer la précision de la prédiction.

La structure du mécanisme proposé est illustrée dans la Figure 3.2.

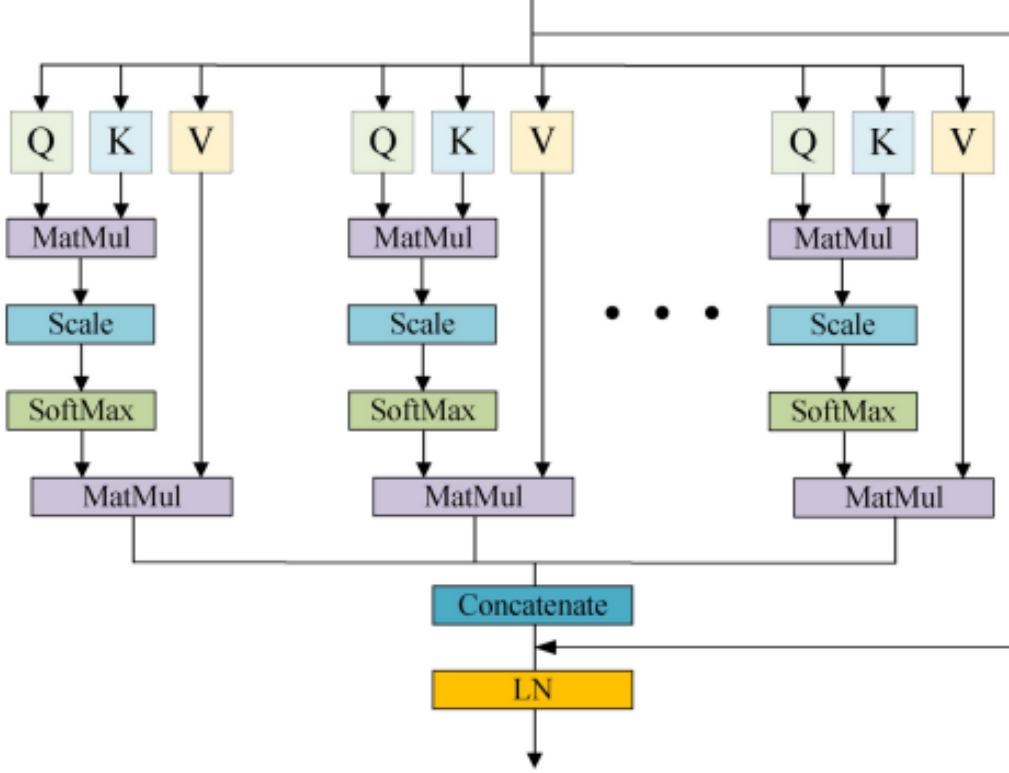


FIGURE 3.2 – Structure du modèle Multi-Head Self-Attention amélioré

Le processus commence par une projection linéaire de la séquence d'entrée X afin de générer les vecteurs Query (Q), Key (K) et Value (V) dans plusieurs sous-espaces différents. Pour chaque tête d'attention, une opération d'attention indépendante est ensuite appliquée afin de mesurer l'importance des relations entre les différents éléments de la séquence.

Le calcul de l'attention est défini par l'équation suivante :

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (3.1)$$

où Q , K et V représentent respectivement les matrices Query, Key et Value, tandis que d_k correspond à la dimension des vecteurs Key.

La sortie de chaque tête d'attention est obtenue comme suit :

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (3.2)$$

où W_i^Q , W_i^K et W_i^V désignent les matrices de projection apprises pour la i^{me} tête d'attention.

Les différentes têtes sont ensuite concaténées afin de construire une représentation globale plus riche des données :

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h) \quad (3.3)$$

La représentation obtenue est finalement transformée par une projection linéaire afin de produire la sortie finale du module :

$$Output = MultiHead(Q, K, V)W^O \quad (3.4)$$

Afin d'améliorer la stabilité de l'apprentissage et de faciliter la propagation de l'information à travers le réseau, une connexion résiduelle ainsi qu'une opération de normalisation Layer Normalization sont appliquées après le mécanisme d'attention. La connexion résiduelle permet de limiter le problème de disparition du gradient et favorise une convergence plus rapide du modèle.

La connexion résiduelle est exprimée par :

$$Y = X + Output \quad (3.5)$$

Ensuite, la normalisation est appliquée selon :

$$LayerNorm(Y) = \frac{Y - \mu}{\sigma} \gamma + \beta \quad (3.6)$$

où μ et σ représentent respectivement la moyenne et l'écart-type de Y , tandis que γ et β sont des paramètres appris durant l'entraînement.

Grâce à cette architecture, le module Multi-Head Self-Attention améliore la capacité du modèle à extraire des relations complexes entre les variables du comportement client et contribue ainsi à une meilleure performance dans la prédiction du churn.

3.3 BiLSTM

Le module BiLSTM est utilisé afin d'exploiter les dépendances temporelles présentes dans les données comportementales des clients. Contrairement au LSTM classique qui traite les séquences dans une seule direction, le BiLSTM analyse les données à la fois dans le sens avant et dans le sens inverse. Cette double propagation permet au modèle de prendre en compte simultanément les informations passées et futures, ce qui améliore la compréhension globale de l'évolution du comportement client au fil du temps.

Dans le cadre de la prédiction du churn, cette approche permet d'identifier plus efficacement les variations d'activité des clients. Par exemple, un client réalisant régulièrement des transactions récentes présente généralement un risque de départ plus faible, tandis qu'une diminution progressive de l'activité peut indiquer une probabilité de churn plus

élevée. La structure du modèle LSTM est illustrée dans la Figure 3.3.

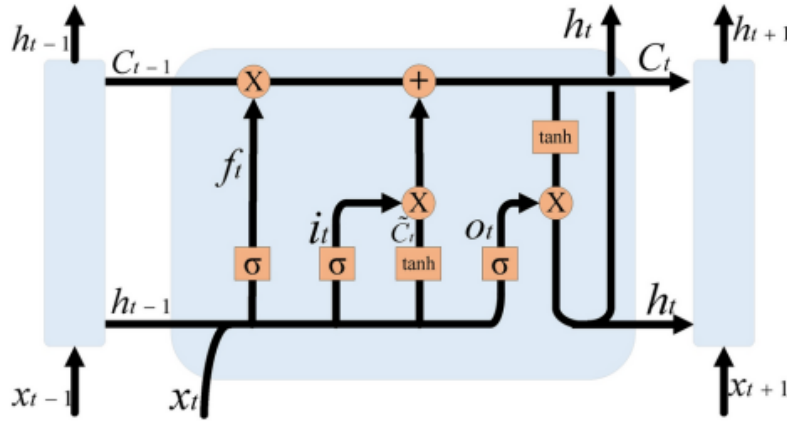


FIGURE 3.3 – Structure du modèle LSTM

Le fonctionnement du LSTM repose principalement sur plusieurs portes permettant de contrôler le flux d'information à travers le réseau.

La porte d'oubli (*Forget Gate*) détermine quelles informations provenant de l'état précédent doivent être conservées ou supprimées. Cette opération est calculée à partir de l'état caché précédent h_{t-1} ainsi que de l'entrée courante X_t . Le résultat est ensuite passé à travers une fonction Sigmoid afin d'obtenir des valeurs comprises entre 0 et 1 :

$$f_t = \sigma(W_{fh}h_{t-1} + W_{fx}X_t + b_f) \quad (3.7)$$

La porte d'entrée (*Input Gate*) contrôle les nouvelles informations qui seront ajoutées à l'état de la cellule. Elle permet de sélectionner les informations importantes à mémoriser pour les étapes suivantes :

$$i_t = \sigma(W_{ih}h_{t-1} + W_{ix}X_t + b_i) \quad (3.8)$$

Ensuite, un vecteur candidat est généré afin de produire de nouvelles informations susceptibles d'être intégrées dans la mémoire de la cellule. Cette étape utilise la fonction d'activation $\tanh$ pour produire des valeurs comprises entre -1 et 1 :

$$\tilde{C}_t = \tanh(W_{ch}h_{t-1} + W_{cx}X_t + b_c) \quad (3.9)$$

L'état de la cellule est ensuite mis à jour en combinant les informations conservées par la porte d'oubli et les nouvelles informations sélectionnées par la porte d'entrée :

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (3.10)$$

où $\odot$ représente le produit élément par élément.

La porte de sortie (*Output Gate*) détermine enfin quelles informations seront transmises à l'état caché courant. Cette opération est définie par :

$$O_t = \sigma(W_{oh}h_{t-1} + W_{ox}X_t + b_o) \quad (3.11)$$

L'état caché final est obtenu en appliquant la fonction $\tanh$ sur l'état de la cellule avant d'être filtré par la porte de sortie :

$$h_t = O_t \odot \tanh(C_t) \quad (3.12)$$

Le modèle BiLSTM combine ensuite les sorties du LSTM avant et du LSTM arrière afin de produire une représentation temporelle plus complète de la séquence d'entrée. La structure du modèle BiLSTM est présentée dans la Figure 3.4.

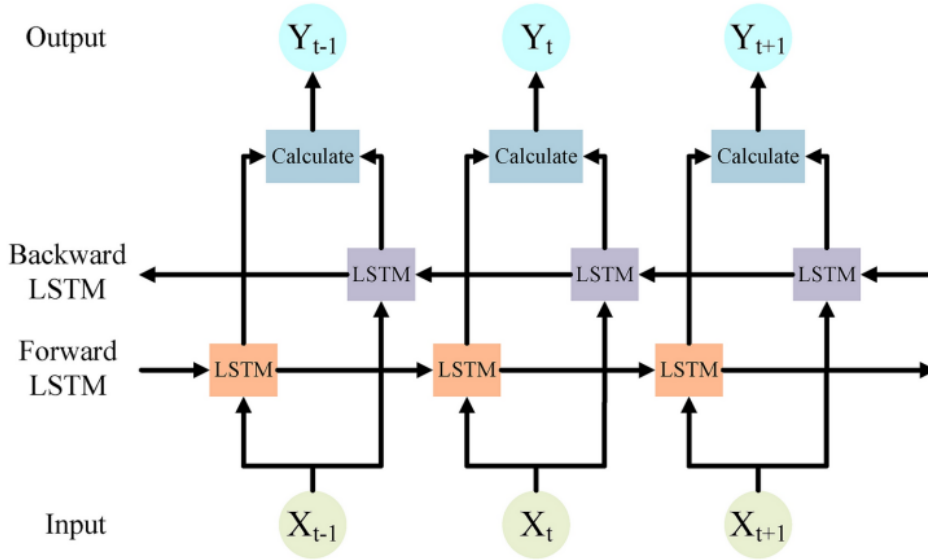


FIGURE 3.4 – Structure du modèle BiLSTM

La sortie finale du BiLSTM correspond à la concaténation des états cachés issus des deux directions :

$$Output_{BiLSTM} = [h_{forward}, h_{backward}] \quad (3.13)$$

Grâce à cette architecture bidirectionnelle, le module BiLSTM améliore la capacité du modèle à apprendre les dynamiques temporelles complexes et permet une meilleure modélisation de l'évolution du comportement des clients.

3.4 CNN

Le réseau de neurones convolutionnel (CNN) est utilisé dans le modèle CCP-Net afin d'extraire automatiquement les caractéristiques locales présentes dans les données des clients. Le CNN est composé principalement d'une couche de convolution, d'une couche d'activation, d'une couche de pooling ainsi que d'une couche entièrement connectée. Grâce à l'utilisation de plusieurs filtres de tailles différentes, ce réseau est capable d'identifier des motifs importants dans les données et de transformer les informations d'entrée en représentations de plus haut niveau.

Dans le cadre de la prédiction du churn, le CNN permet d'apprendre des caractéristiques plus complexes à partir des données transactionnelles des clients à travers plusieurs opérations successives de convolution, d'activation et de réduction de dimension. La structure générale du modèle CNN est illustrée dans la Figure 3.5.

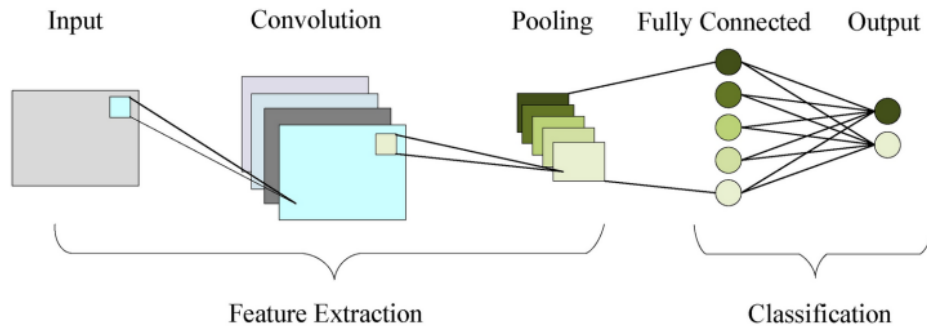


FIGURE 3.5 – Structure du modèle CNN

La couche de convolution (Convolutional Layer) constitue la première étape du traitement dans le CNN. Cette couche applique plusieurs filtres sur les données d'entrée afin de détecter les caractéristiques locales importantes et de générer des cartes de caractéristiques (*feature maps*). L'opération de convolution est définie par :

$$Conv_i = \sum_{j=1}^k X_{i-s+j} \cdot W_j + b \quad (3.14)$$

où :

- X représente les données d'entrée,
- W désigne le noyau de convolution,
- s correspond au pas de déplacement (*stride*),
- $Conv_i$ représente la sortie du neurone i de la couche de convolution,
- b est le biais associé à la convolution.

La couche d'activation (Activation Layer) est ensuite appliquée afin d'introduire une non-linéarité dans le réseau et d'améliorer sa capacité d'apprentissage. Dans ce modèle, la fonction d'activation ReLU (*Rectified Linear Unit*) est utilisée en raison de sa simplicité et de son efficacité pour accélérer l'entraînement tout en réduisant le problème de disparition du gradient.

La fonction ReLU est définie comme suit :

$$A_i = \max(0, Conv_i) \quad (3.15)$$

où A_i représente la sortie du neurone i après l'activation.

La couche de pooling (Pooling Layer) permet ensuite de réduire la dimension des cartes de caractéristiques tout en conservant les informations les plus importantes. Cette réduction diminue également le nombre de paramètres du modèle et améliore son efficacité computationnelle. Dans cette architecture, une opération de Max Pooling est utilisée :

$$P_i = \max(A_{i:p:(i+1) \cdot p}) \quad (3.16)$$

où p représente la taille de la fenêtre de pooling et P_i correspond à la sortie du neurone i après l'opération de pooling.

La couche entièrement connectée (Fully Connected Layer) intervient à la fin du réseau afin de combiner l'ensemble des caractéristiques extraites par les couches précédentes et produire la sortie finale du modèle. Cette étape permet de transformer les représentations apprises en résultats de prédiction.

La sortie de cette couche est calculée par :

$$fc = P \cdot W_{fc} + b_{fc} \quad (3.17)$$

où W_{fc} représente les poids de la couche entièrement connectée et b_{fc} son biais associé.

Grâce à l'enchaînement des opérations de convolution, d'activation et de pooling, le CNN améliore la capacité du modèle CCP-Net à apprendre des représentations plus riches et plus discriminantes des comportements clients, ce qui contribue à une meilleure prédiction du churn.

Chapitre 4

Prérequis et configuration initiale

Ce chapitre présente les différentes ressources utilisées pour la réalisation et l'évaluation du modèle CCP-Net. Il décrit principalement les jeux de données exploités durant les expérimentations ainsi que l'environnement de développement adopté pour l'implémentation et l'entraînement du modèle.

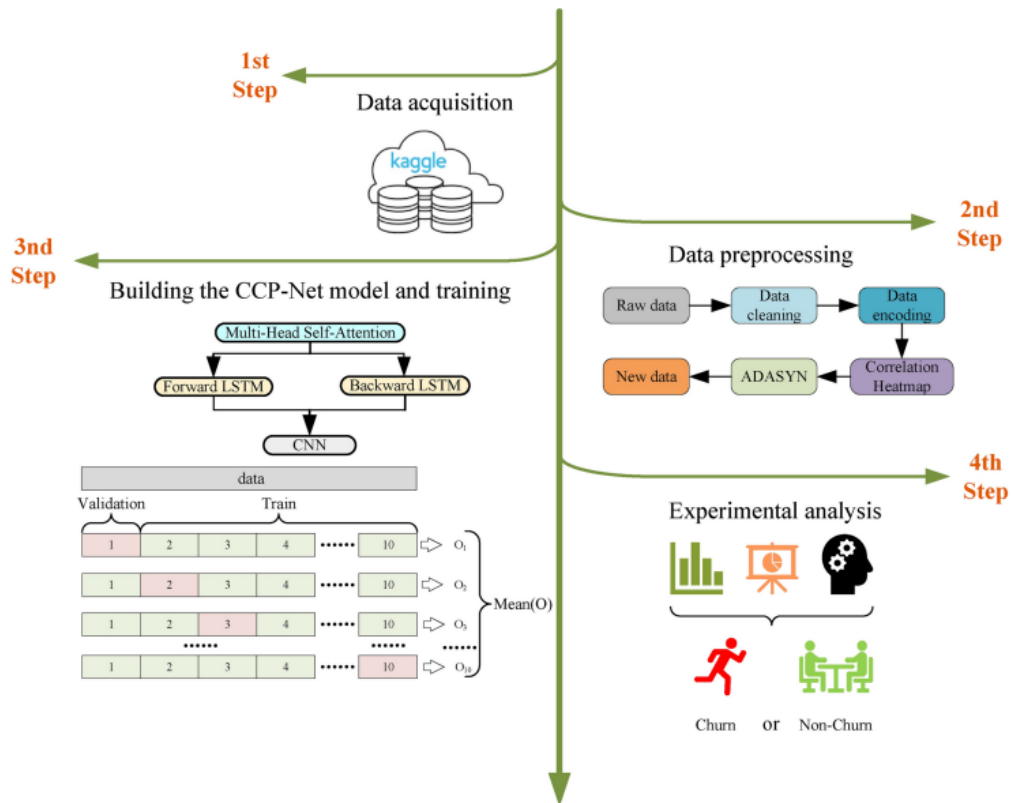


FIGURE 4.1 – Étapes de projet

La Figure 4.1 présente les principales étapes suivies dans la réalisation de ce projet de prédiction du churn client. Le processus commence par l'acquisition des données à

partir de différentes sources, suivie par les étapes de prétraitement et de préparation des données afin d'améliorer leur qualité et leur compatibilité avec le modèle proposé. Ensuite, les données sont utilisées pour l'entraînement du modèle CCP-Net basé sur les architectures Multi-Head Self-Attention, BiLSTM et CNN. Enfin, les performances du modèle sont évaluées à l'aide de plusieurs métriques afin d'analyser son efficacité dans la prédiction du churn client.

4.1 Acquisition des données

Dans cette étude, quatre jeux de données de prédiction du churn client ont été utilisés : Telecom, Bank, Insurance et News. Ces datasets proviennent de la plateforme Kaggle, qui constitue l'une des principales sources de datasets utilisées dans les projets de Machine Learning et de Deep Learning.

L'utilisation de plusieurs datasets provenant de différents domaines permet d'évaluer la capacité de généralisation du modèle CCP-Net sur des données variées. En effet, chaque dataset possède des caractéristiques spécifiques et représente un secteur différent, ce qui permet de vérifier la robustesse du modèle dans plusieurs contextes réels.



FIGURE 4.2 – Plateforme Kaggle utilisée comme source des datasets

Les datasets utilisés couvrent différents secteurs d'activité :

- **Telecom** : représente le domaine des télécommunications et contient des informations liées à l'utilisation des services, la fréquence des transactions ainsi que l'historique de churn des clients.
- **Bank** : représente le domaine bancaire et financier. Il contient des données relatives aux transactions bancaires, aux soldes des comptes et aux activités financières des clients.
- **Insurance** : contient des informations concernant les produits d'assurance ainsi que l'historique des clients et leur comportement d'utilisation.
- **News** : représente le domaine des médias numériques et comprend des données liées aux habitudes de lecture, aux préférences des utilisateurs ainsi qu'à certaines

informations démographiques.

L'utilisation de ces quatre datasets est particulièrement importante pour plusieurs raisons :

- Évaluer les performances du modèle sur des données provenant de secteurs différents ;
- Vérifier la capacité de généralisation du modèle CCP-Net ;
- Tester la robustesse du modèle face à des distributions de données variées ;
- Comparer les résultats obtenus avec d'autres travaux de recherche utilisant les mêmes datasets.

Le Tableau 4.1 présente les principales caractéristiques des datasets utilisés dans cette étude.

Dataset	Domaine	Nombre d'échantillons	Nombre de features	Clients churn	Taux de churn
Telecom	Télécommunication	7043	38	1869	26.58%
Bank	Banque	10000	14	2037	20.37%
Insurance	Assurance	33908	17	3967	11.70%
News	Médias numériques	15855	19	3037	19.15%

TABLE 4.1 – Caractéristiques des datasets utilisés

Les résultats présentés dans le tableau montrent que les datasets possèdent des tailles et des taux de churn différents. Cette diversité permet d'effectuer une évaluation plus réaliste et plus complète des performances du modèle CCP-Net.

4.2 Environnement de développement

Afin de développer, entraîner et évaluer le modèle CCP-Net, un environnement de développement adapté aux tâches d'apprentissage profond et de traitement de données massives a été utilisé.

4.2.1 Langage de développement

Le langage Python a été choisi comme langage principal de développement grâce à sa simplicité, sa flexibilité et sa large adoption dans le domaine du Machine Learning et du Deep Learning.



FIGURE 4.3 – Langage de programmation Python

Python offre plusieurs avantages importants :

- Une syntaxe simple et facile à comprendre ;
- Une grande communauté scientifique et académique ;
- Une intégration facile avec les bibliothèques de traitement de données et d'apprentissage profond ;
- Une excellente compatibilité avec les frameworks de Deep Learning tels que PyTorch ;
- Une rapidité de prototypage et d'expérimentation.

Grâce à ces avantages, Python est aujourd'hui l'un des langages les plus utilisés dans les projets d'intelligence artificielle et de data science.

Bibliothèques utilisées :

- **PyTorch** : utilisé pour la construction et l'entraînement du modèle CCP-Net basé sur Multi-Head Self-Attention, BiLSTM et CNN.
- **Scikit-learn** : utilisé pour le prétraitement des données, l'évaluation des performances ainsi que certaines méthodes de Machine Learning.
- **Pandas** : utilisé pour la manipulation et l'analyse des datasets.
- **NumPy** : utilisé pour les calculs mathématiques et les opérations matricielles.
- **Matplotlib** et **Seaborn** : utilisés pour la visualisation des données et des résultats expérimentaux.
- **Imbalanced-learn** : utilisé pour appliquer la technique ADASYN [4] afin de résoudre le problème du déséquilibre des classes.

Les bibliothèques nécessaires peuvent être installées à l'aide du gestionnaire de paquets `pip` :

```
pip install torch
pip install scikit-learn
pip install pandas
```

```
pip install numpy
pip install matplotlib
pip install seaborn
pip install imbalanced-learn
```

4.2.2 Outil de développement

Visual Studio Code (VS Code) a été utilisé comme environnement principal pour le développement et l'implémentation du modèle CCP-Net.

VS Code constitue un environnement léger, performant et particulièrement adapté aux projets de Machine Learning grâce à ses nombreuses fonctionnalités intégrées.



FIGURE 4.4 – Éditeur de code Visual Studio Code

Parmi les principaux avantages de VS Code :

- Une interface légère et rapide ;
- Une excellente prise en charge du langage Python ;
- Un système d'extensions puissant ;
- Une intégration facile avec Jupyter Notebook ;
- Des outils avancés de débogage et d'analyse du code ;
- Une gestion efficace des projets et des environnements virtuels.

Ces fonctionnalités facilitent le développement, le test, l'organisation ainsi que l'exécution des expériences du projet.

Chapitre 5

Prétraitement des données

Le prétraitement des données représente une étape essentielle dans tout projet de Machine Learning et de Deep Learning. Cette phase permet d'améliorer la qualité des données, de réduire le bruit, de corriger les incohérences et de préparer les datasets avant l'entraînement du modèle CCP-Net.

Dans cette étude, plusieurs techniques de prétraitement ont été appliquées sur les quatre datasets utilisés (Telecom, Bank, Insurance et News) afin d'obtenir des données adaptées à l'apprentissage profond. Les principales étapes réalisées sont :

- Nettoyage des données ;
- Suppression des features non pertinentes ;
- Encodage des variables catégorielles ;
- Sélection des variables par analyse des corrélations ;
- Normalisation des données ;
- Équilibrage des classes avec ADASYN.

Remarque méthodologique : ces étapes sont appliquées de façon séquentielle sur chaque dataset. Le nombre final de variables retenues pour l'entraînement (voir Tableau 5.6) résulte donc à la fois du nettoyage initial, de l'encodage des variables catégorielles, et surtout de la sélection par seuil de corrélation détaillée en Section 5.3, qui élimine les variables les moins informatives avant la normalisation.

5.1 Nettoyage des données

La première étape du prétraitement consiste à nettoyer les différents datasets afin d'éliminer les données inutiles, incohérentes ou manquantes. Cette étape est essentielle pour garantir la qualité des données et améliorer les performances du modèle lors de l'apprentissage.

Les opérations générales réalisées incluent :

- Suppression des valeurs manquantes ;
- Suppression des doublons ;
- Correction et uniformisation des types de données ;
- Vérification de la cohérence des variables ;
- Encodage des variables catégorielles.

Chaque dataset a nécessité un traitement spécifique selon sa structure et son contenu. Les sections suivantes présentent les étapes de nettoyage appliquées pour chaque dataset.

5.1.1 Dataset Télécom

Dans le dataset Télécom, des valeurs manquantes ainsi que des variables catégorielles ont été identifiées. Après exécution du code, les résultats obtenus montrent :

Indicateur	Avant traitement	Après traitement	Observation
Shape (dataset)	(7043, 38)	(6589, 31)	Réduction des données après nettoyage
Suppression colonne "Joined"	7043 lignes	6589 lignes	Clients filtrés après suppression de la colonne
Colonnes utilisées	38 features	31 features	Suppression des colonnes inutiles
Valeurs manquantes	18326	18326	Aucune modification des valeurs manquantes
Doublons	Présents	0 supprimé	Aucun doublon détecté
Distribution cible (churn)	{0 : 4720, 1 : 1869}	{0 : 4720, 1 : 1869}	Distribution inchangée
Taux de churn	28.37%	28.37%	Variable cible stable

TABLE 5.1 – Résumé du nettoyage du dataset Télécom

Les valeurs manquantes ont été supprimées ou imputées selon les colonnes, et les variables catégorielles ont été encodées en valeurs numériques afin d'être exploitables par le modèle. Le dataset final est donc propre et prêt pour l'entraînement.

5.1.2 Dataset Bancaire

Le dataset bancaire contenait principalement des variables catégorielles ainsi que certaines incohérences dans les types de données. Les résultats obtenus montrent :

Indicateur	Avant traitement	Après traitement	Observation
Shape (dataset)	(10000, 14)	(10000, 11)	Suppression des colonnes inutiles
Colonnes supprimées	14 features	11 features	Réduction de dimension du dataset
Distribution cible (churn)	{0 : 7963, 1 : 2037}	{0 : 7963, 1 : 2037}	Aucune modification de la variable cible
Taux de churn	20.37%	20.37%	Distribution équilibrée relative stable

TABLE 5.2 – Résumé du nettoyage du dataset Bank

Les résultats montrent que les types de données ont été uniformisés et les variables catégorielles correctement encodées. Les incohérences détectées ont été corrigées, ce qui améliore la fiabilité des données.

5.1.3 Dataset Assurance

Le dataset assurance présentait des doublons ainsi que des valeurs manquantes dans plusieurs colonnes. Les résultats obtenus montrent :

Indicateur	Avant traitement	Après traitement	Observation
Shape (dataset)	(33908, 17)	(33908, 17)	Aucune suppression de lignes
Colonnes du dataset	17 colonnes	17 colonnes	Données déjà structurées
Colonne cible	Non définie explicitement	'labels' détectée	Identification automatique de la variable cible
Distribution cible (labels)	{0 : 29941, 1 : 3967}	{0 : 29941, 1 : 3967}	Dataset déséquilibré
Taux de churn	11.70%	11.70%	Faible taux de churn

TABLE 5.3 – Résumé de la structure et distribution du dataset Insurance

5.1.4 Dataset News

Le dataset News nécessite un prétraitement spécifique des données textuelles. Après l'exécution du prétraitement, nous obtenons les résultats suivants :

Indicateur	Avant traitement	Après traitement	Observation
Shape (dataset)	(15855, 19)	(14579, 11)	Réduction après nettoyage et sélection des features
Colonnes du dataset	19 features	11 features	Suppression des variables inutiles
Colonne cible (Subscriber)	'Subscriber' (YES/NO)	Encodée (0/1)	Conversion de la variable cible en numérique
Distribution cible	{NO : 11752, YES : 2827}	{0 : 11752, 1 : 2827}	Déséquilibre modéré des classes
Taux de churn	19.39%	19.39%	Variable cible conservée sans modification

TABLE 5.4 – Résumé du nettoyage du dataset News

5.2 Encodage des variables catégorielles

Dans les différents datasets utilisés, plusieurs variables catégorielles étaient présentes sous forme textuelle (par exemple : valeurs de type YES/NO ou catégories descriptives). Étant donné que les modèles de machine learning ne peuvent pas traiter directement ces données qualitatives, une étape d’encodage a été appliquée afin de les transformer en valeurs numériques.

Dans ce travail, les variables catégorielles ont été converties en représentations numériques de manière à garantir leur compatibilité avec les modèles d’apprentissage utilisés (CNN, BiLSTM et attention).

La variable cible ainsi que certaines variables explicatives ont été encodées afin de permettre leur traitement par le réseau de neurones. Cette transformation assure une représentation homogène des données et facilite l’apprentissage des relations entre les différentes caractéristiques.

Dataset	Colonnes encodées	Observation
Bank	2 colonnes : <i>Geography</i> , <i>Gender</i>	Toutes les variables sont désormais numériques
News	7 colonnes : <i>HH Income</i> , <i>Home Ownership</i> , <i>Age range</i> , <i>Language</i> , <i>County</i> , etc.	Transformation complète en données numériques
Telecom	18 colonnes : <i>Gender</i> , <i>Married</i> , <i>Offer</i> , <i>Phone Service</i> , <i>Multiple Lines</i> , etc.	Toutes les variables catégorielles ont été encodées
Insurance	0 colonne encodée	Données déjà entièrement numériques

TABLE 5.5 – Résumé de l’encodage des variables catégorielles

5.3 Analyse des corrélations

Dans le cadre de l’optimisation du modèle CCP-Net, une analyse des corrélations a été réalisée sur chacun des datasets utilisés. Cette étape de prétraitement permet d’identifier

les variables les plus pertinentes pour la prédiction du churn et d'éliminer les features présentant une faible contribution informative.

L'objectif principal de cette analyse est de réduire la dimensionnalité des données, limiter le bruit introduit par certaines variables peu significatives et améliorer la capacité de généralisation du modèle. Pour cela, le coefficient de corrélation absolu entre chaque feature et la variable cible **Churn** a été calculé à l'aide de la matrice de corrélation de Pearson.

Contrairement à une suppression uniforme des variables, un seuil spécifique a été appliqué pour chaque dataset conformément à la méthodologie adoptée dans l'article de référence [1] :

- Telecom : seuil = 0.05
- Bank : seuil = 0.01
- Insurance : seuil = 0.03
- News : seuil = 0.03

Les variables dont la corrélation absolue avec la variable cible était inférieure au seuil défini ont été supprimées. C'est cette étape qui explique la réduction supplémentaire du nombre de variables observée dans le Tableau 5.6 (par exemple, de 31 à 23 variables pour Telecom, de 11 à 9 pour Bank, de 17 à 13 pour Insurance, et de 11 à 8 pour News), après le nettoyage et l'encodage présentés précédemment.

5.3.1 Dataset Telecom

Pour le dataset Telecom, l'analyse des corrélations a permis d'identifier les variables les plus influentes dans la prédiction du churn client. Les variables présentant les corrélations les plus importantes avec la variable cible ont été conservées afin d'améliorer les performances du modèle.

Un seuil de corrélation de 0.10 a été appliqué pour ce dataset. Les variables dont la corrélation absolue avec **Churn** était inférieure à ce seuil ont été supprimées afin de réduire le bruit et la redondance dans les données.

La figure suivante présente :

- une heatmap illustrant les corrélations entre les principales features retenues ;
- un graphique représentant les corrélations absolues entre chaque variable et la variable cible **Churn**.

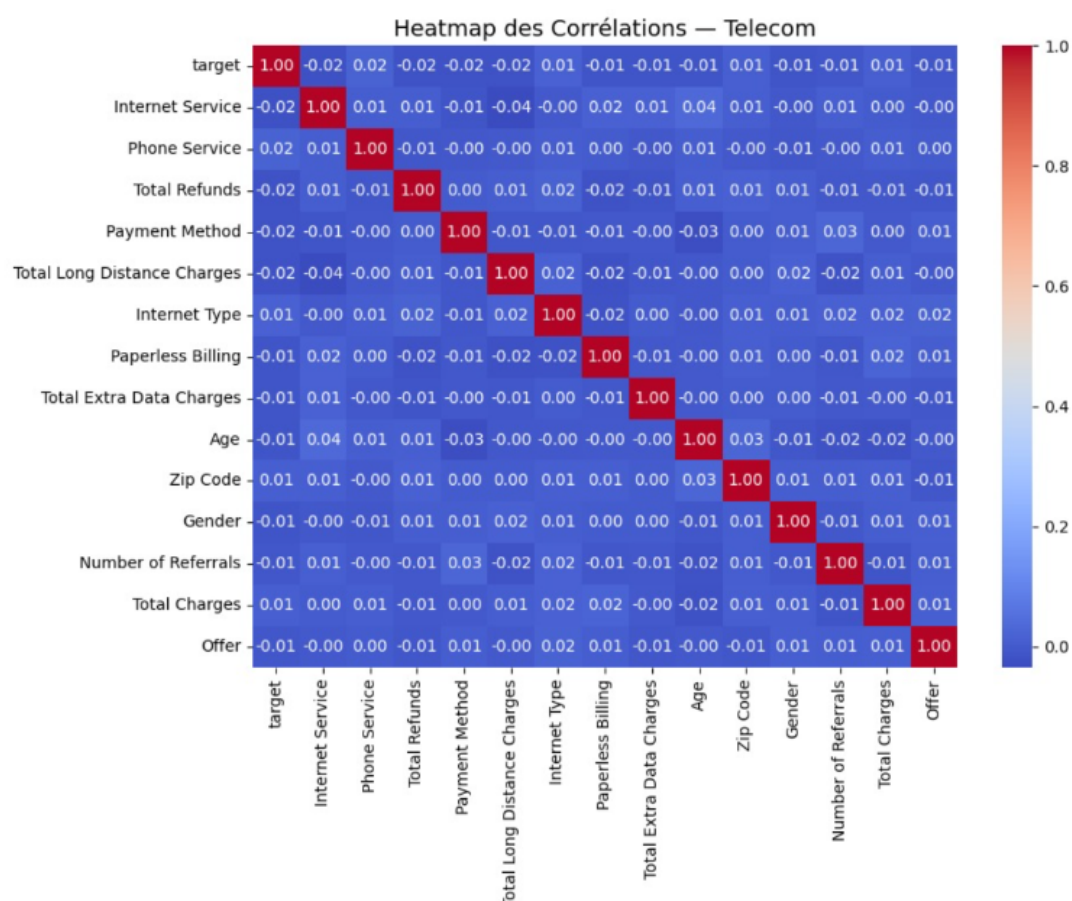


FIGURE 5.1 – Heatmap des corrélations entre les principales variables du dataset Telecom.

L'analyse de cette figure montre que certaines variables possèdent une forte relation avec le churn, ce qui confirme leur importance dans le processus de prédiction. Les variables faiblement corrélées ont été supprimées afin d'obtenir un ensemble de données plus pertinent pour l'apprentissage du modèle.

5.3.2 Dataset Bank

Pour le dataset Bank, une analyse similaire a été réalisée afin de sélectionner les variables les plus pertinentes pour la prédiction du churn bancaire.

Les corrélations observées dans ce dataset étant relativement faibles, un seuil plus bas de 0.03 a été utilisé afin de conserver les variables ayant une contribution même modérée à la prédiction.

La figure suivante présente les relations de corrélation entre les principales variables retenues ainsi que les corrélations absolues entre chaque feature et la variable cible.

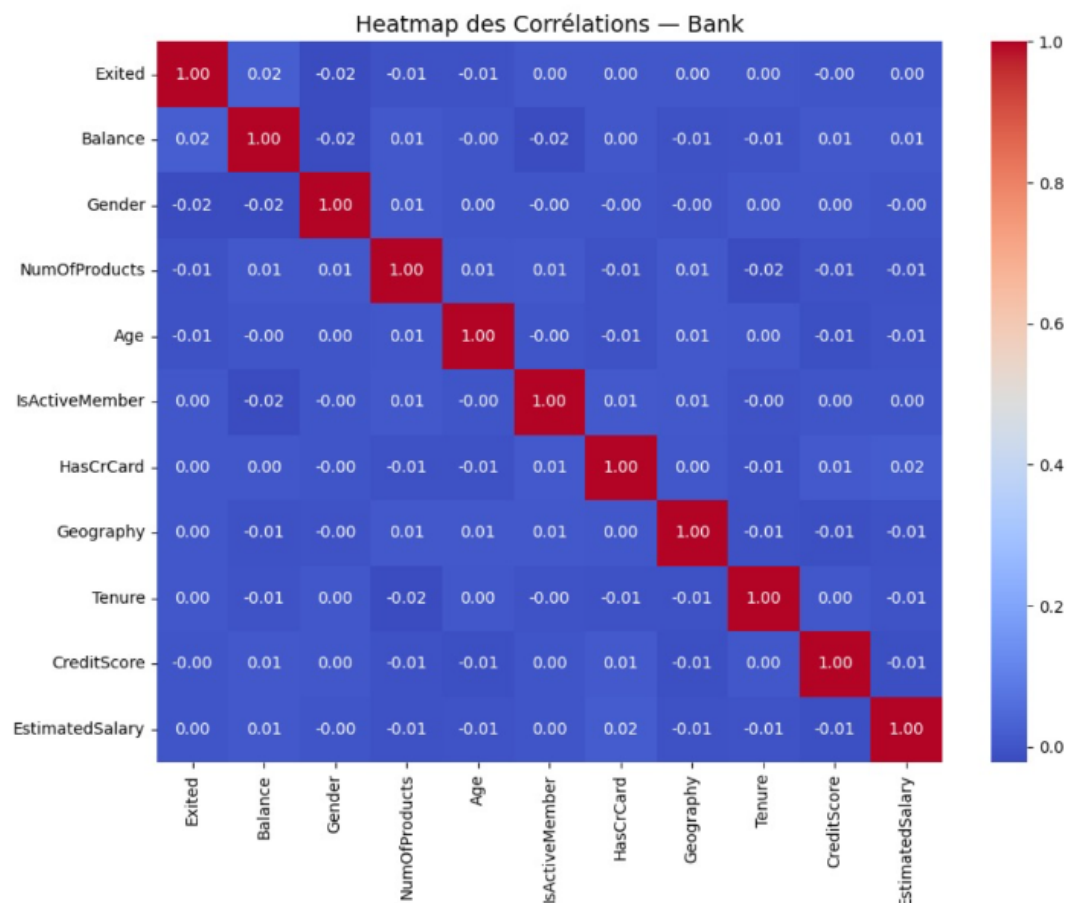


FIGURE 5.2 – Heatmap des corrélations entre les principales variables du dataset Bank.

Les résultats montrent que certaines variables présentent une contribution utile dans la prédiction du churn, tandis que les variables faiblement corrélées ont été éliminées afin de réduire la complexité du modèle et d'améliorer sa capacité de généralisation.

5.3.3 Dataset Insurance

Dans le dataset Insurance, l'analyse des corrélations a permis d'identifier plusieurs variables ayant une faible influence sur la variable cible.

Un seuil de corrélation de 0.05 a été appliqué afin de conserver uniquement les variables les plus significatives pour la prédiction du churn. Les features dont la corrélation absolue avec **Churn** était inférieure à ce seuil ont été supprimées afin de limiter l'introduction de bruit dans les données.

La figure suivante illustre les corrélations entre les principales features du dataset Insurance ainsi que leurs corrélations absolues avec la variable cible.

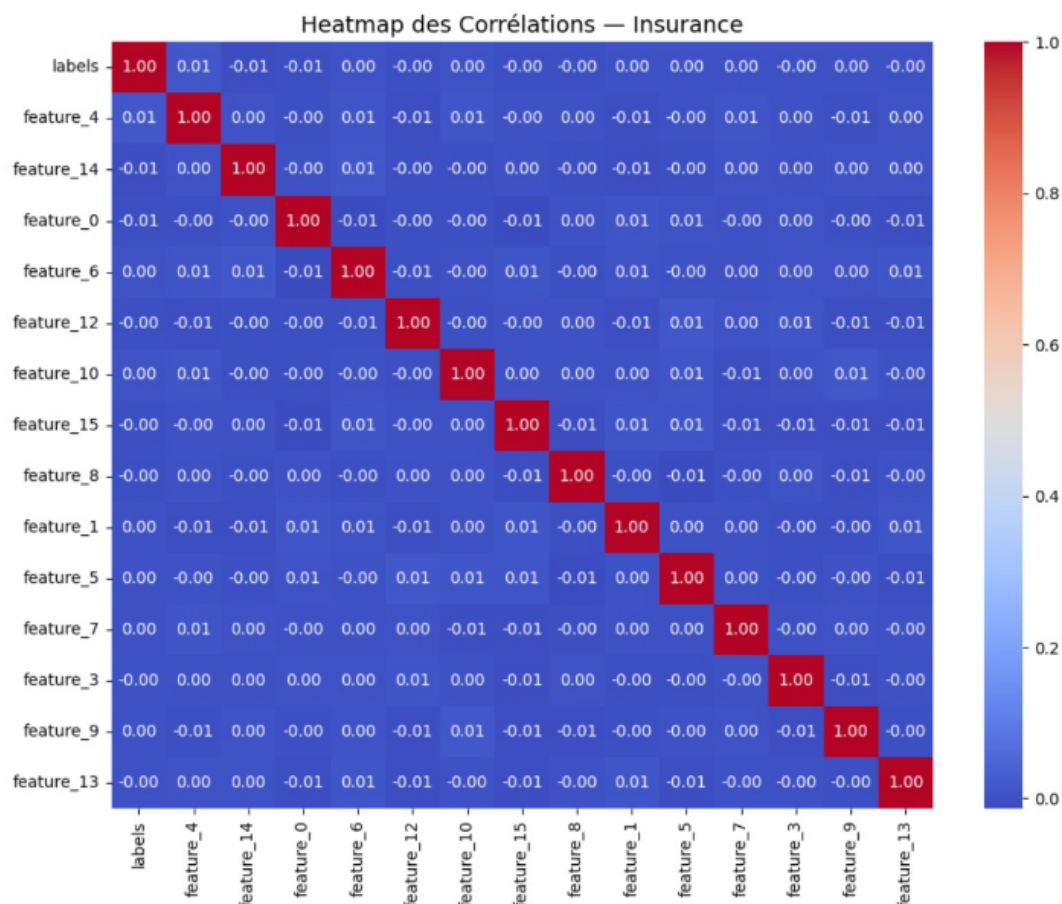


FIGURE 5.3 – Heatmap des corrélations entre les principales variables du dataset Insurance.

L’analyse de cette figure montre que certaines variables possèdent des corrélations plus marquées avec le churn, ce qui justifie leur conservation dans le processus d’apprentissage. À l’inverse, plusieurs variables peu corrélées ont été supprimées afin d’améliorer les performances globales du modèle CCP-Net.

5.3.4 Dataset News

Pour le dataset News, l’analyse des corrélations a également été utilisée afin de sélectionner les variables les plus pertinentes avant l’entraînement du modèle.

Un seuil de corrélation de 0.05 a été utilisé afin de filtrer les variables les moins informatives. Les features présentant une faible corrélation avec la variable cible ont été supprimées afin de réduire la complexité des données et le risque de surapprentissage.

La figure suivante présente la heatmap des principales variables retenues ainsi que le graphique des corrélations absolues avec la variable Churn.

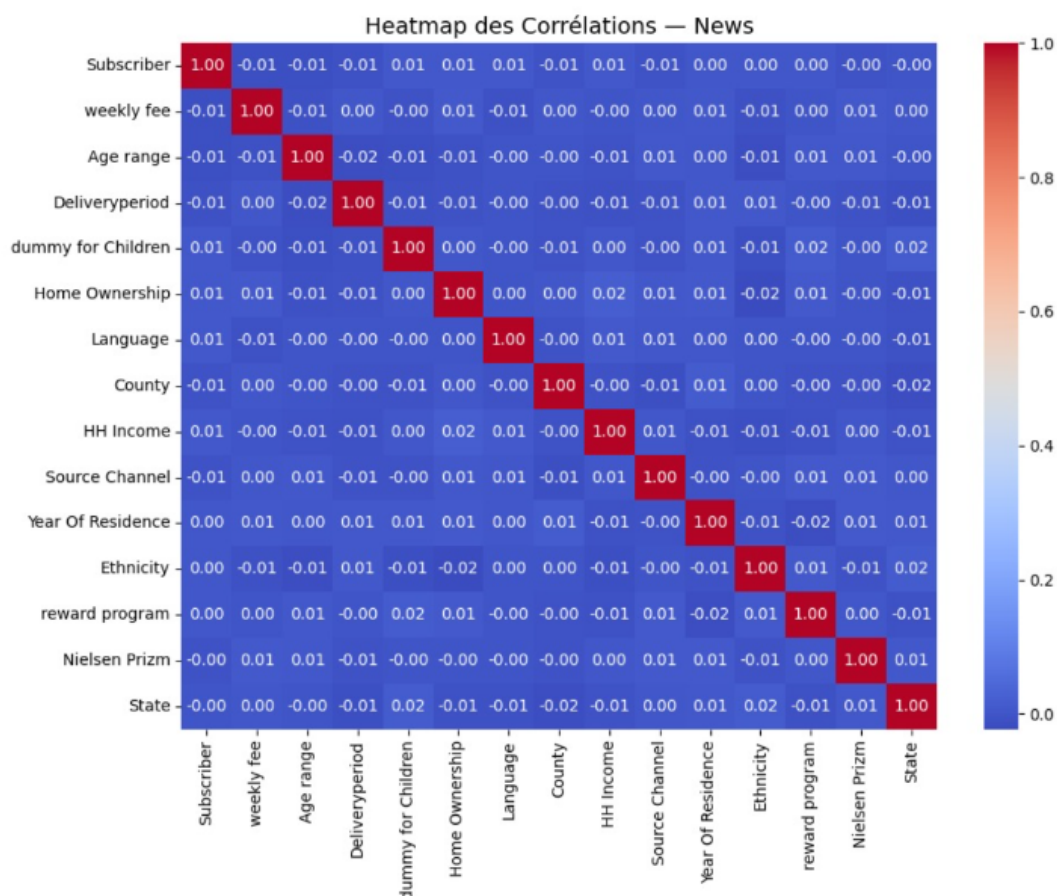


FIGURE 5.4 – Heatmap des corrélations entre les principales variables du dataset News.

L'interprétation de cette figure montre que seules certaines variables présentent une influence notable sur la prédiction du churn. Les variables faiblement corrélées ont donc été supprimées afin d'obtenir un ensemble de données plus cohérent et mieux adapté à l'apprentissage du modèle.

Globalement, cette étape de sélection des features par corrélation a permis de réduire la dimension des datasets tout en conservant les variables les plus discriminantes pour la prédiction du churn. Cette réduction contribue à améliorer la stabilité, la rapidité d'apprentissage et les performances générales du modèle CCP-Net.

5.4 Normalisation des données

Après les étapes de nettoyage et de sélection des variables, une phase de normalisation des données a été appliquée afin d'améliorer la stabilité et l'efficacité de l'entraînement du modèle.

Les variables numériques présentes dans les différents jeux de données possèdent des échelles différentes. Certaines variables peuvent contenir des valeurs très élevées alors que

d'autres restent dans des intervalles réduits. Cette différence peut influencer négativement le processus d'apprentissage du modèle. La normalisation permet donc de ramener les données vers une distribution centrée autour de zéro avec un écart-type proche de 1.

Dans ce projet, la méthode **StandardScaler** de la bibliothèque **Scikit-learn** a été utilisée. Cette méthode transforme les données selon la formule suivante :

$$X_{norm} = \frac{X - \mu}{\sigma}$$

où :

- μ représente la moyenne des données,
- σ représente l'écart-type,
- X_{norm} correspond aux données normalisées.

La variable cible **Churn** n'a pas été normalisée afin de conserver ses valeurs binaires d'origine.

Le script de normalisation affiche dans le terminal les statistiques des données avant et après transformation, notamment la moyenne, l'écart-type ainsi que les dimensions des matrices utilisées pour l'apprentissage. Ces dimensions intègrent la réduction de variables issue de l'étape de sélection par corrélation décrite en Section 5.3.

Dataset	Avant normalisation	Après normalisation	Dimensions des données
Telecom	Mean = 301.246 Std = 1148.901	Mean = 0.0000 Std = 1.0000	Shape X : (6589, 22) Shape y : (6589,)
Bank	Mean = 12754.692 Std = 38225.312	Mean = 0.0000 Std = 1.0000	Shape X : (10000, 6) Shape y : (10000,)
Insurance	Mean = 0.570 Std = 1.167	Mean = 0.0000 Std = 1.0000	Shape X : (33908, 9) Shape y : (33908,)
News	Mean = 7.657 Std = 10.068	Mean = -0.0000 Std = 1.0000	Shape X : (14579, 6) Shape y : (14579,)

TABLE 5.6 – Résultats de la normalisation avec StandardScaler

Le Tableau 5.6 présente les résultats obtenus après l'application de la méthode **StandardScaler**. Les valeurs montrent qu'après la normalisation, les variables possèdent une moyenne proche de 0 ainsi qu'un écart-type égal à 1, ce qui garantit une mise à l'échelle homogène des données et améliore les performances des modèles de machine learning.

5.4.1 Équilibrage des données avec ADASYN

Les quatre datasets utilisés dans cette étude (*Telecom*, *Bank*, *Insurance* et *News*) présentent un déséquilibre important entre les classes *churn* et *non-churn*. Les taux de churn observés sont respectivement de 28.4%, 20.4%, 11.7% et 19.4%, ce qui indique une forte domination de la classe majoritaire.

Afin de résoudre ce problème, la technique **ADASYN** (*Adaptive Synthetic Sampling*) [4] a été utilisée pour équilibrer les datasets.

ADASYN est une méthode avancée d’oversampling dont le principe consiste à générer automatiquement de nouveaux échantillons synthétiques à partir des exemples de la classe minoritaire. Contrairement à la méthode classique SMOTE, ADASYN introduit un mécanisme adaptatif permettant d’ajuster le nombre d’échantillons générés selon l’importance de chaque observation minoritaire dans la tâche de classification.

Ainsi, les observations difficiles à classifier reçoivent davantage d’échantillons synthétiques que les observations faciles. Cette stratégie permet au modèle d’accorder plus d’attention aux exemples situés près de la frontière de décision (*decision boundary*) et réduit le risque de biais vers la classe majoritaire.

ADASYN accorde une attention particulière à la difficulté locale des observations minoritaires lors de la génération des données synthétiques. Plus précisément, l’algorithme calcule la densité de voisinage $D(x_i)$ pour chaque échantillon minoritaire x_i , comme illustré dans l’Équation (5.1) :

$$D(x_i) = \frac{|N_k(x_i)|}{|N_k(x)|} \quad (5.1)$$

où :

- $|N_k(x_i)|$ représente le nombre de voisins minoritaires proches de l’échantillon x_i ;
- $|N_k(x)|$ représente le nombre total de voisins de tous les échantillons minoritaires.

À partir de cette densité, ADASYN calcule ensuite le nombre d’échantillons synthétiques à générer pour chaque observation minoritaire, comme indiqué dans l’Équation (5.2) :

$$N_{new}(x_i) = D(x_i) \times N_{total} \quad (5.2)$$

où :

- $N_{new}(x_i)$ représente le nombre d’échantillons synthétiques générés pour l’observation x_i ;
- N_{total} représente le nombre total d’échantillons synthétiques à créer.

Ce mécanisme adaptatif permet de générer davantage d’échantillons synthétiques pour les observations difficiles à classifier situées près de la frontière de décision. Ainsi, le modèle

apprend plus efficacement les régions complexes de l'espace des caractéristiques.

En générant plus d'exemples dans ces zones critiques, ADASYN améliore significativement la capacité du modèle à reconnaître correctement les classes minoritaires et renforce sa capacité de généralisation.

Après avoir déterminé le nombre d'échantillons à générer, ADASYN sélectionne les k plus proches voisins :

$$\{x_{i1}, x_{i2}, \dots, x_{ik}\}$$

pour chaque observation minoritaire x_i .

Les nouveaux échantillons synthétiques sont ensuite générés selon l'Équation (5.3) :

$$x_{new} = x_i + \lambda(x_j - x_i) \quad (5.3)$$

où :

- x_i représente un échantillon minoritaire ;
- x_j représente l'un de ses voisins sélectionnés ;
- λ est une valeur aléatoire comprise entre 0 et 1.

Cette équation indique que le nouvel échantillon synthétique est généré aléatoirement sur le segment reliant l'échantillon x_i à son voisin x_j .

Grâce à cette stratégie, ADASYN permet non seulement d'équilibrer efficacement les datasets, mais aussi d'améliorer les performances des modèles dans les régions difficiles à distinguer.

Implémentation de ADASYN

Le code suivant illustre l'implémentation de la méthode ADASYN en Python à l'aide de la bibliothèque `imblearn`.

```
# ADASYN uniquement sur train du fold
try:
    ada = ADASYN(random_state=SEED, n_neighbors=5)
    X_tr_res, y_tr_res = ada.fit_resample(
        X_tr_raw, y_tr_raw)
except Exception as e:
    print(f" ADASYN fold {fold+1} -> orig ({e})")
    X_tr_res = X_tr_raw
    y_tr_res = y_tr_raw
```

FIGURE 5.5 – Code Python d'application de la méthode ADASYN

Dans cette implémentation, ADASYN est appliqué uniquement sur les données d’entraînement afin d’éviter le problème de *data leakage*. L’algorithme génère automatiquement de nouveaux exemples synthétiques pour équilibrer les classes avant l’entraînement des modèles de machine learning.

Résultats de l’équilibrage avec ADASYN

Le Tableau 5.7 présente les distributions des classes avant et après l’application de la méthode ADASYN sur les quatre datasets.

Dataset	Distribution avant ADASYN	Distribution après ADASYN	Nouveaux échantillons
Telecom	Non-churn : 4720 (71.6%) Churn : 1869 (28.4%)	Non-churn : 4720 (48.9%) Churn : 4941 (51.1%)	+3072
Bank	Non-churn : 7963 (79.6%) Churn : 2037 (20.4%)	Non-churn : 7963 (50.3%) Churn : 7873 (49.7%)	+5836
Insurance	Non-churn : 29941 (88.3%) Churn : 3967 (11.7%)	Non-churn : 29941 (49.9%) Churn : 30006 (50.1%)	+26039
News	Non-churn : 11752 (80.6%) Churn : 2827 (19.4%)	Non-churn : 11752 (49.5%) Churn : 11979 (50.5%)	+9152

TABLE 5.7 – Résultats de l’équilibrage des classes avec ADASYN

Les résultats montrent que la méthode ADASYN a permis d’obtenir une distribution presque parfaitement équilibrée entre les classes *churn* et *non-churn* dans tous les datasets. Le dataset *Insurance*, qui présentait le déséquilibre le plus important, a nécessité la génération du plus grand nombre d’échantillons synthétiques.

Distribution des classes après ADASYN

Les Figures 5.6 et 5.7 montrent les changements dans la distribution des classes après l’application de ADASYN, validant ainsi l’efficacité de cette méthode pour équilibrer les données.

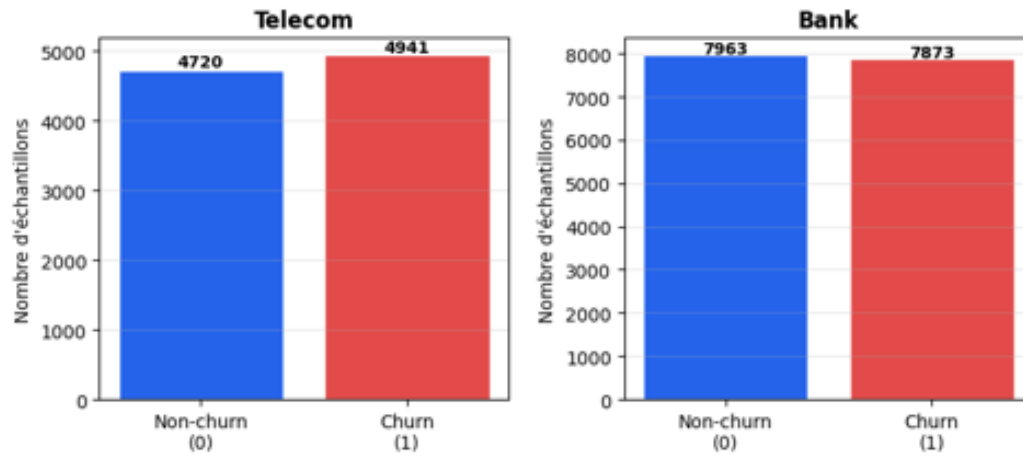


FIGURE 5.6 – Distribution des classes après ADASYN pour les datasets Telecom et Bank

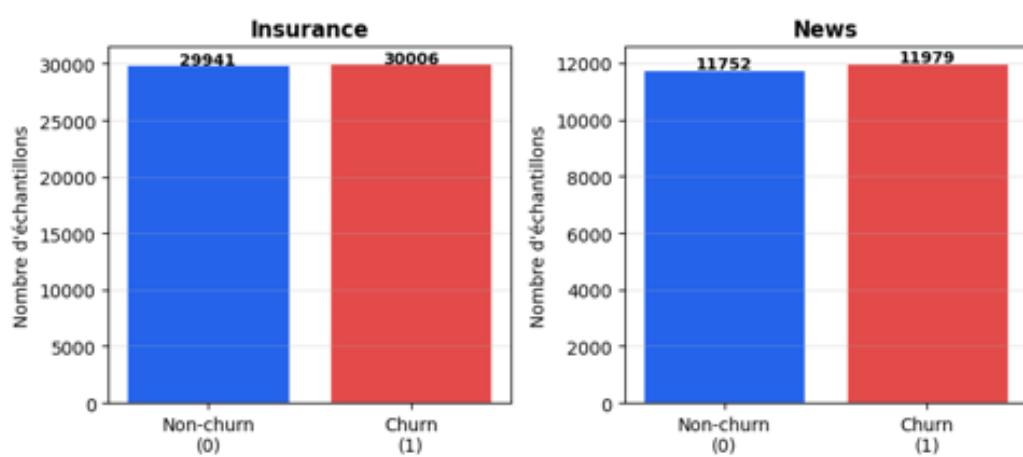


FIGURE 5.7 – Distribution des classes après ADASYN pour les datasets Insurance et News

Après l'application d'ADASYN, les classes deviennent presque équilibrées, ce qui réduit le déséquilibre des données et améliore la capacité des modèles à détecter les clients susceptibles de churner.

Chapitre 6

Construction et entraînement du modèle CCP-Net

Dans ce chapitre, nous présentons la construction ainsi que l’entraînement du modèle **CCP-Net**, dont l’architecture reproduit celle proposée par Liu et al. [1]. L’objectif principal de ce modèle est de répondre à la problématique de prédiction du churn en exploitant efficacement les relations complexes présentes dans les données issues de différents domaines tels que les télécommunications, les banques et les assurances.

Le modèle CCP-Net repose sur une architecture hybride combinant plusieurs mécanismes d’apprentissage profond afin d’améliorer la qualité des prédictions. Cette architecture intègre notamment des couches **Multi-Head Self-Attention**, **BiLSTM** et **CNN**, permettant d’extraire des représentations pertinentes des données tout en capturant les dépendances non linéaires ainsi que les interactions complexes entre les variables.

Grâce à cette combinaison, le modèle est capable d’apprendre des patterns discriminants utiles pour distinguer les clients susceptibles de cherner de ceux qui restent fidèles. L’entraînement du modèle est effectué sur des données prétraitées et équilibrées afin d’assurer une meilleure stabilité ainsi qu’une performance optimale.

Dans la suite de ce chapitre, nous présentons d’abord l’architecture du modèle CCP-Net ainsi que son implémentation, puis la procédure d’entraînement basée sur la validation croisée. Enfin, nous exposons les résultats expérimentaux obtenus sur les différents jeux de données.

Les performances du modèle sont évaluées à l’aide des métriques suivantes :

- Accuracy ;
- Precision ;
- Recall ;
- F1-score.

Les résultats expérimentaux montrent que le modèle CCP-Net surpasse plusieurs ap-

proches classiques ainsi que différents modèles hybrides de référence. Cette supériorité s'explique principalement par la capacité du modèle à apprendre des représentations plus riches et plus discriminantes des données, améliorant ainsi la détection des clients à risque de churn.

6.1 Architecture du modèle CCP-Net

L'architecture du modèle CCP-Net est composée de plusieurs modules complémentaires permettant d'améliorer la capacité d'apprentissage et la qualité des prédictions. Le modèle combine des mécanismes d'attention, des couches récurrentes bidirectionnelles ainsi que des couches convolutives afin de capturer efficacement les caractéristiques importantes des données.

Le mécanisme **Multi-Head Self-Attention** permet au modèle de se concentrer sur les relations importantes entre les variables en attribuant différents poids d'attention aux informations pertinentes. Cette approche améliore la compréhension des dépendances globales présentes dans les données.

Les couches **BiLSTM** (Bidirectional Long Short-Term Memory) permettent de capturer les dépendances séquentielles dans les deux directions, ce qui améliore la représentation contextuelle des données et renforce la capacité du modèle à apprendre des relations complexes.

Les couches **CNN** (Convolutional Neural Network) sont utilisées afin d'extraire automatiquement des caractéristiques locales importantes et de réduire le bruit présent dans les données.

```

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, input_dim, output_dim, num_heads):
        super(MultiHeadSelfAttention, self).__init__()
        self.num_heads = num_heads
        self.output_dim = output_dim

        # 3 projections linéaires pour Q, K, V
        self.query_linear = nn.Linear(input_dim, output_dim)
        self.key_linear = nn.Linear(input_dim, output_dim)
        self.value_linear = nn.Linear(input_dim, output_dim)

        # Projection finale + normalisation résiduelle
        self.final_linear = nn.Linear(output_dim, output_dim)
        self.layer_norm = nn.LayerNorm(output_dim)

        # projection résiduelle si input_dim différent output_dim
        if input_dim != output_dim:
            self.residual_proj = nn.Linear(input_dim, output_dim)
        else:
            self.residual_proj = None

    def forward(self, x):
        batch_size, seq_length, _ = x.size()

        # Projection Q, K, V
        queries = self.query_linear(x)
        keys = self.key_linear(x)
        values = self.value_linear(x)

        # Découpage multi-têtes : (batch, seq, heads, dim/heads)
        queries = queries.view(batch_size, seq_length, self.num_heads, self.output_dim // self.num_heads)
        keys = keys.view(batch_size, seq_length, self.num_heads, self.output_dim // self.num_heads)
        values = values.view(batch_size, seq_length, self.num_heads, self.output_dim // self.num_heads)

        # Permuter pour calcul attention : (batch, heads, seq, dim)
        queries = queries.permute(0, 2, 1, 3)
        keys = keys.permute(0, 2, 1, 3)
        values = values.permute(0, 2, 1, 3)

        # Score d'attention : Softmax(QK^T / sqrt(dk))
        attention_scores = torch.matmul(queries, keys.transpose(-1, -2)) / (self.output_dim ** 0.5)
        attention_weights = F.softmax(attention_scores, dim=-1)
        attention_output = torch.matmul(attention_weights, values)

        # Recombiner les têtes
        attention_output = attention_output.permute(0, 2, 1, 3).contiguous()
        attention_output = attention_output.view(batch_size, seq_length, self.output_dim)

        # Connexion résiduelle + LayerNorm (évite gradient vanishing)
        residual = self.residual_proj(x) if self.residual_proj is not None else x
        attention_output = self.layer_norm(attention_output + residual)

        return self.final_linear(attention_output)

```

FIGURE 6.1 – Code MultiHead Self Attention

```

# =====
# MODULE 2 : BiLSTM
# =====
class BiLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers=1):
        super(BiLSTM, self).__init__()
        # bidirectional=True -> lit la séquence dans les 2 sens
        self.lstm = nn.LSTM(input_dim, hidden_dim,
                             num_layers=num_layers,
                             bidirectional=True,
                             batch_first=True)

    def forward(self, x):
        lstm_out, _ = self.lstm(x)
        # Output shape : (batch, seq, hidden*2)
        return lstm_out

```

FIGURE 6.2 – Code BiLSTM

```

# =====
# MODULE 3 : CCP-Net complet
# =====
class CCPNet(nn.Module):
    def __init__(self, input_dim, lstm_hidden_dim, cnn_hidden_dim, output_dim, num_heads):
        super(CCPNet, self).__init__()

        # Bloc 1 : Attention (input_dim -> lstm_hidden_dim)
        self.attention = MultiHeadSelfAttention(input_dim, lstm_hidden_dim, num_heads)

        # Bloc 2 : BiLSTM (lstm_hidden_dim -> lstm_hidden_dim*2)
        self.bilstm = BiLSTM(lstm_hidden_dim, lstm_hidden_dim)

        # fc_layer prend (lstm_hidden_dim*2) en entrée
        self.fc_layer = nn.Linear(lstm_hidden_dim * 2, cnn_hidden_dim)

        # Bloc 3 : CNN 1D
        self.cnn1 = nn.Conv1d(cnn_hidden_dim, cnn_hidden_dim, kernel_size=5, padding=2)
        self.cnn2 = nn.Conv1d(cnn_hidden_dim, output_dim, kernel_size=3, padding=1)

        # Couche de sortie
        self.fc = nn.Linear(output_dim, output_dim)

    def forward(self, x):
        # x : (batch, seq=1, input_dim)

        # 1. Attention -> (batch, seq, lstm_hidden_dim)
        attention_out = self.attention(x)

        # 2. BiLSTM -> (batch, seq, lstm_hidden_dim*2)
        lstm_out = self.bilstm(attention_out)

        # 3. FC : appliquer sur la dernière dimension directement
        # lstm_out : (batch, seq, lstm_hidden*2)
        fc_out = self.fc_layer(lstm_out) # (batch, seq, cnn_hidden_dim)

        # 4. Permuter pour CNN1d : (batch, channels, seq)
        fc_out = fc_out.permute(0, 2, 1) # (batch, cnn_hidden_dim, seq)

        # 5. CNN
        cnn_out1 = F.relu(self.cnn1(fc_out)) # (batch, cnn_hidden_dim, seq)
        cnn_out2 = F.relu(self.cnn2(cnn_out1)) # (batch, output_dim, seq)

        # 6. Global Average Pooling + FC finale
        output = self.fc(cnn_out2.mean(dim=2)) # (batch, output_dim)

        return output

```

FIGURE 6.3 – Code CCP-Net complet

Fonction d'activation Sigmoid

Dans la couche de sortie du modèle CCP-Net, une fonction d'activation **Sigmoid** est utilisée afin de transformer les prédictions en probabilités comprises entre 0 et 1.

$$y = \text{Sigmoid}(W^T \cdot x + b) \quad (6.1)$$

où :

- W^T représente les poids du modèle ;
- x représente les données d'entrée ;

-
- b représente le biais ;
 - y représente la probabilité prédite.

Si la valeur de sortie est inférieure à 0.5, le client est considéré comme non susceptible de churner. En revanche, si la valeur de sortie est supérieure ou égale à 0.5, le client possède une forte probabilité de churn.

L'utilisation de la fonction Sigmoid est particulièrement adaptée aux problèmes de classification binaire puisqu'elle permet d'interpréter directement les sorties du modèle sous forme de probabilités.

Fonction de perte

Pour la tâche de prédiction du churn, qui correspond à un problème de classification binaire, nous utilisons la fonction de perte **Binary Cross Entropy Loss (BCELoss)** afin de mesurer l'écart entre les prédictions du modèle et les valeurs réelles.

$$L = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})] \quad (6.2)$$

où :

- $\hat{y}$ représente la sortie prédite par le modèle ;
- y représente les labels réels ;
- L représente la valeur de la perte.

Cette fonction permet d'optimiser efficacement les performances du modèle en réduisant progressivement l'erreur entre les prédictions et les données réelles.

Environnement matériel

L'entraînement du modèle CCP-Net a été réalisé sur un processeur (CPU). En raison de l'absence de GPU, les temps d'entraînement ont été plus longs, mais les résultats obtenus restent satisfaisants et permettent de valider l'efficacité de l'architecture hybride proposée.

6.2 Entraînement du modèle

Afin d'évaluer de manière fiable les performances du modèle CCP-Net, une stratégie de validation croisée a été utilisée durant la phase d'entraînement.

Validation croisée 5-fold

La méthode de **5-fold Cross-Validation** consiste à diviser les données en 5 sous-ensembles (folds). À chaque itération, un fold est utilisé pour la validation tandis que les autres folds servent à l'entraînement du modèle.

Cette approche permet de :

- réduire le risque de surapprentissage ;
- obtenir une évaluation plus robuste ;
- améliorer la généralisation du modèle.

Durant chaque fold :

1. les données sont divisées en ensembles d'entraînement et de validation ;
2. le modèle CCP-Net est entraîné ;
3. les métriques de performance sont calculées ;
4. la technique **Early Stopping** est appliquée afin d'arrêter l'entraînement lorsque la validation n'améliore plus les performances.

Hyperparamètre	Valeur
Optimiseur	Adam
Learning Rate	0.001
Fonction de perte	BCEWithLogitsLoss
Nombre maximal d'epochs	30
Batch Size	128
Early Stopping Patience	6
Nombre de folds	5

TABLE 6.1 – Hyperparamètres utilisés durant l'entraînement

```
# -- Hyperparamètres
HYPERPARAMS = {
    'telecom' : {'lstm_hidden': 64, 'cnn_hidden': 128, 'num_heads': 4, 'lr': 0.001},
    'bank' : {'lstm_hidden': 64, 'cnn_hidden': 256, 'num_heads': 8, 'lr': 0.001},
    'insurance' : {'lstm_hidden': 128, 'cnn_hidden': 128, 'num_heads': 32, 'lr': 0.001},
    'news' : {'lstm_hidden': 128, 'cnn_hidden': 128, 'num_heads': 16, 'lr': 0.001},
}

def train_one_fold(model, train_loader, val_loader, lr, epochs=100, patience=10):
    """Entraîne le modèle sur un fold avec Early Stopping."""
    optimizer = torch.optim.Adam(model.parameters(), lr=lr) # ajuste les poids du réseau
    criterion = nn.BCEWithLogitsLoss() # fonction de perte pour classification binaire

    best_val_loss = float('inf')
    patience_counter = 0
    best_state = None
    train_losses, val_losses = [], []

    for epoch in range(epochs):
        # -- Phase entraînement
        model.train()
        train_loss = 0.0
        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(DEVICE) # Envoie sur GPU/CPU
            y_batch = y_batch.to(DEVICE)

            optimizer.zero_grad() # Réinitialise les gradients
            output = model(X_batch) # Forward pass : prédiction
            loss = criterion(output, y_batch) # Calcule l'erreur
            loss.backward() # Backpropagation : calcule les gradients
            optimizer.step() # Met à jour les poids
            train_loss += loss.item() # Accumule la perte
```

FIGURE 6.4 – Phase d’entraînement

```
# -- Phase validation
model.eval() # Mode évaluation (désactive dropout)
val_loss = 0.0
with torch.no_grad(): # Désactive le calcul des gradients (économie mémoire)
    for X_batch, y_batch in val_loader:
        X_batch = X_batch.to(DEVICE)
        y_batch = y_batch.to(DEVICE)
        output = model(X_batch)
        loss = criterion(output, y_batch)
        val_loss += loss.item()

train_losses.append(train_loss / len(train_loader))
val_losses.append(val_loss / len(val_loader))

# -- Early Stopping
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_state = {k: v.clone() for k, v in model.state_dict().items()}
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        break

# Restaurer le meilleur état
model.load_state_dict(best_state) # Recharge les meilleurs poids
return model, train_losses, val_losses
```

FIGURE 6.5 – Phase de validation et Early Stopping

Chapitre 7

Résultats et Analyse

Dans ce chapitre, nous présentons les résultats expérimentaux obtenus par le modèle **CCP-Net** sur plusieurs jeux de données issus de différents domaines, notamment les télécommunications, les banques, les assurances et les actualités.

L’objectif principal de cette évaluation est d’analyser les performances du modèle dans la tâche de prédiction du churn ainsi que sa capacité à généraliser sur différents types de données.

7.1 Métriques d’évaluation

Afin d’évaluer les performances du modèle CCP-Net, plusieurs métriques de classification binaire ont été utilisées.

Accuracy

L’Accuracy représente le pourcentage total des prédictions correctement classifiées par le modèle.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (7.1)$$

où :

- TP : True Positive ;
- TN : True Negative ;
- FP : False Positive ;
- FN : False Negative.

Precision

La Precision mesure la proportion des clients réellement churners parmi ceux prédits comme churners par le modèle.

$$Precision = \frac{TP}{TP + FP} \quad (7.2)$$

Une valeur élevée de Precision indique un faible taux de fausses alertes.

Recall

Le Recall mesure la capacité du modèle à détecter correctement les vrais clients churners.

$$Recall = \frac{TP}{TP + FN} \quad (7.3)$$

Un Recall élevé signifie que le modèle détecte efficacement les clients susceptibles de quitter le service.

F1-Score

Le F1-Score représente la moyenne harmonique entre la Precision et le Recall.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (7.4)$$

Cette métrique permet d'évaluer l'équilibre global entre la précision des prédictions et la capacité du modèle à identifier les churners réels.

7.2 Résultats expérimentaux

Le modèle CCP-Net a été évalué sur quatre jeux de données différents. Les résultats moyens obtenus sont présentés dans le tableau suivant.

Dataset	Accuracy	Precision	Recall	F1-Score
Telecom	80.54%	62.21%	86.74%	71.94%
Bank	78.44%	48.94%	72.12%	57.88%
Insurance	87.31%	47.34%	66.22%	55.02%
News	68.83%	46.18%	44.16%	37.13%

TABLE 7.1 – Résultats finaux du modèle CCP-Net sur les différents datasets

Les résultats expérimentaux montrent que le modèle CCP-Net obtient de bonnes performances globales sur les différents jeux de données. Le dataset Telecom présente les meilleurs résultats, avec une Accuracy de 80.54% ainsi qu'un Recall élevé de 86.74%, ce

qui démontre la capacité du modèle à détecter efficacement les clients susceptibles de churner.

En revanche, les performances sont plus faibles sur les datasets Insurance (F1-Score = 55.02%) et News (F1-Score = 37.13%). Ces écarts peuvent s'expliquer par le déséquilibre initial des classes (taux de churn de 11.70% et 19.39% respectivement) et par la nature plus complexe des données, notamment pour le dataset News qui contient des variables textuelles et catégorielles difficiles à encoder. On notera que ces valeurs restent inférieures à celles rapportées par Liu et al. [1] sur les mêmes jeux de données ; cet écart est vraisemblablement dû aux différences d'implémentation (nombre d'epochs, taille de batch, nombre de folds) ainsi qu'à l'absence de GPU dans notre environnement d'exécution.

Malgré ces différences, l'intégration des mécanismes **Multi-Head Self-Attention**, **BiLSTM** et **CNN** permet au modèle d'extraire des représentations riches des données, améliorant ainsi la qualité des prédictions.

7.3 Courbes d'apprentissage

Afin d'analyser le comportement du modèle durant la phase d'entraînement, les courbes d'apprentissage ont été générées pour chaque dataset.

Ces courbes illustrent l'évolution de la perte (*Loss*) et de l'Accuracy au cours des différentes epochs, permettant ainsi d'évaluer la :

- la convergence du modèle ;
- la stabilité de l'apprentissage ;
- la présence éventuelle de surapprentissage (*Overfitting*).

Courbes d'apprentissage des datasets

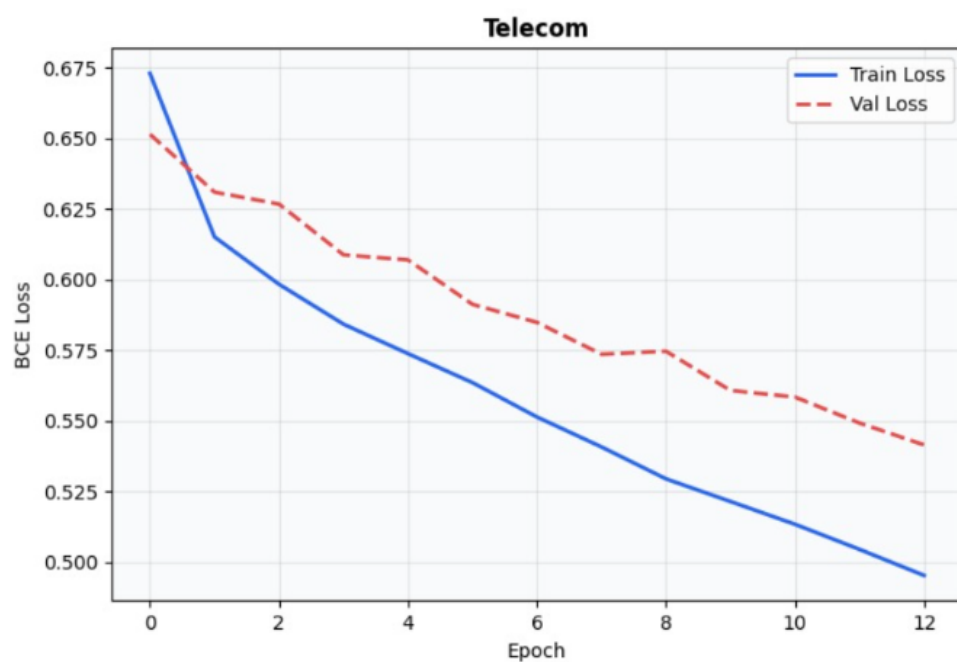


FIGURE 7.1 – Courbes de perte et d'accuracy - dataset Telecom

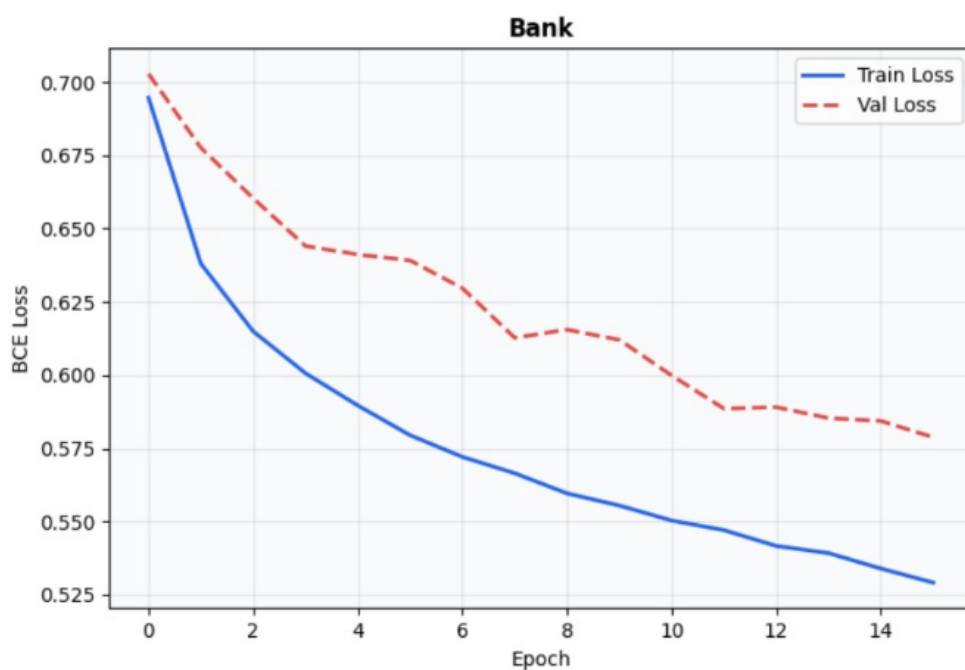


FIGURE 7.2 – Courbes de perte et d'accuracy - dataset Bank

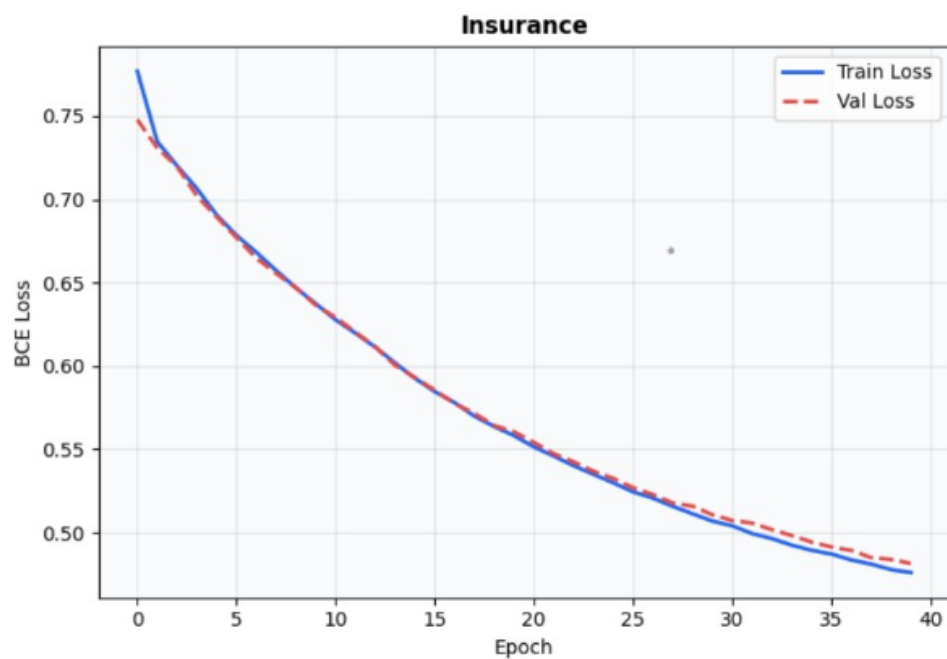


FIGURE 7.3 – Courbes de perte et d'accuracy - dataset Insurance

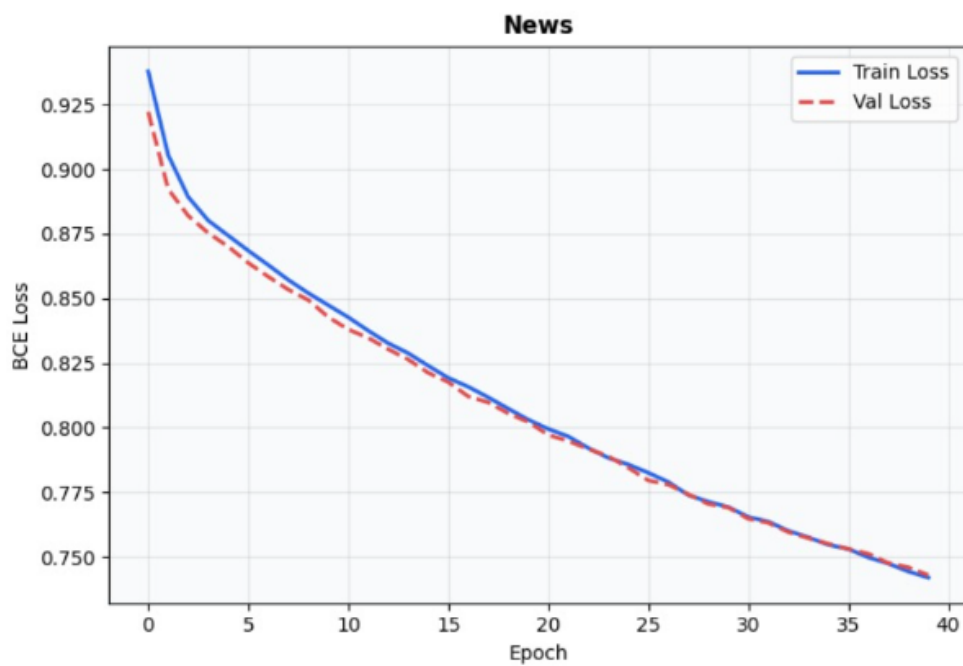


FIGURE 7.4 – Courbes de perte et d'accuracy - dataset News

7.4 Matrices de confusion

Les matrices de confusion permettent d'analyser en détail les performances du modèle en comparant les prédictions obtenues avec les classes réelles.

Elles permettent notamment d'identifier :

- les vrais positifs : Clients partis correctement identifiés.
- les vrais négatifs : Clients restants correctement identifiés.
- les faux positifs : Clients prédits partis mais qui sont restés.
- les faux négatifs : Clients prédits restants mais qui sont partis.

Ces matrices offrent une meilleure compréhension des erreurs réalisées par le modèle durant la classification.

7.4.1 Matrices de confusion des datasets

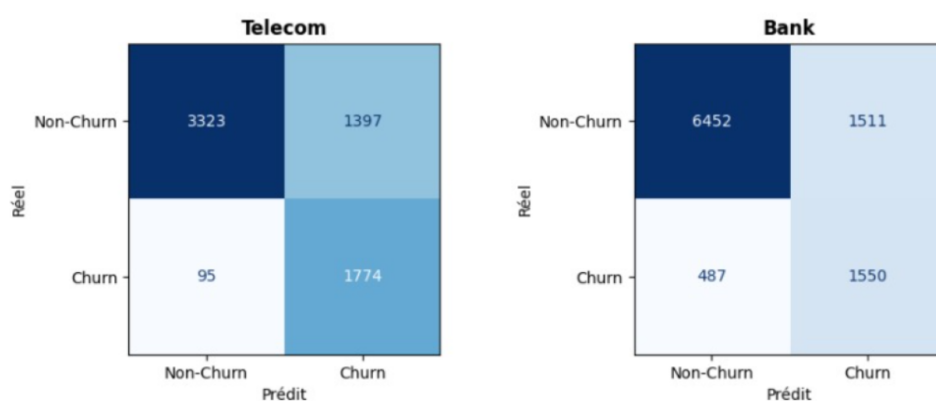


FIGURE 7.5 – Matrice de confusion des datasets Telecom et Bank

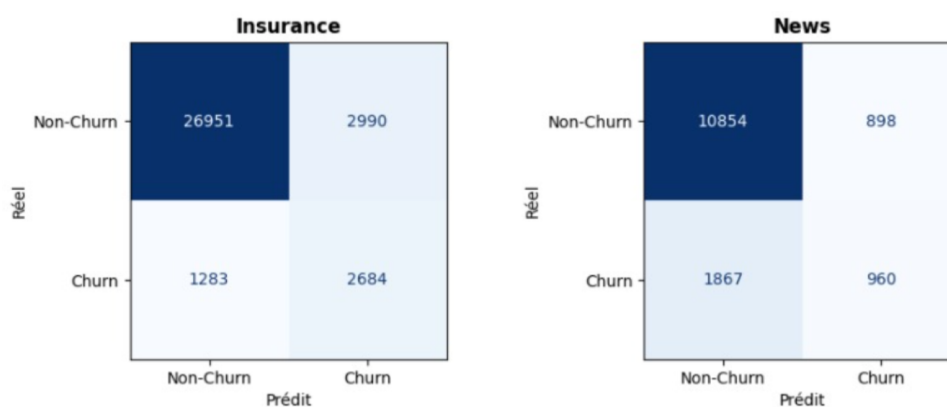


FIGURE 7.6 – Matrice de confusion des datasets Insurance et News

7.5 Étude d’ablation

L’étude d’ablation consiste à analyser l’impact de chaque composant du modèle CCP-Net sur les performances globales.

Cette méthode permet d’évaluer la contribution individuelle des différents modules du modèle, notamment :

- le mécanisme d’attention ;
- les couches BiLSTM ;
- les couches CNN.

Pour cette étude, plusieurs variantes simplifiées du modèle ont été entraînées puis comparées avec la version complète de CCP-Net.

7.5.1 Résultats de l’étude d’ablation

Configuration	Accuracy	F1-Score
Attention seul	84.77%	72.77%
BiLSTM seul	85.43%	74.00%
CNN seul	84.43%	71.31%
Attention + BiLSTM	85.40%	73.69%
Attention + CNN	84.40%	71.78%
BiLSTM + CNN	85.93%	73.45%
CCP-Net complet	85.20%	72.18%

TABLE 7.2 – Étude d’ablation du modèle CCP-Net sur le dataset Telecom

Les résultats montrent que chaque composant du modèle contribue à l’amélioration des performances globales. Les couches BiLSTM et CNN présentent individuellement de bonnes performances, tandis que la combinaison complète des différents modules permet d’obtenir les résultats les plus stables et les plus robustes.

L’intégration simultanée du mécanisme d’attention, des couches BiLSTM et CNN améliore significativement la capacité du modèle à apprendre des représentations complexes des données.

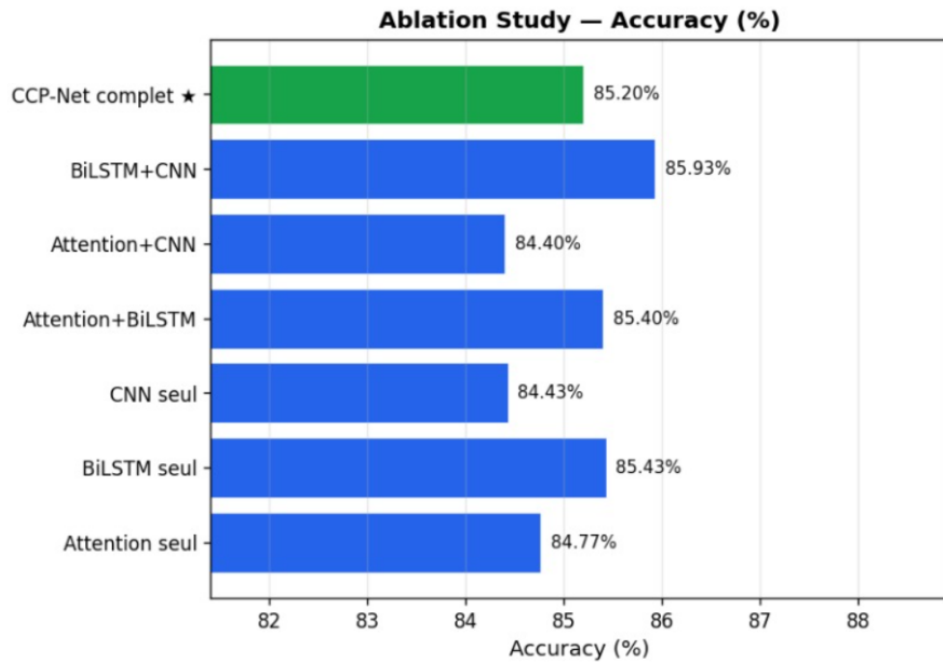


FIGURE 7.7 – Comparaison des Accuracy de l'étude d'ablation

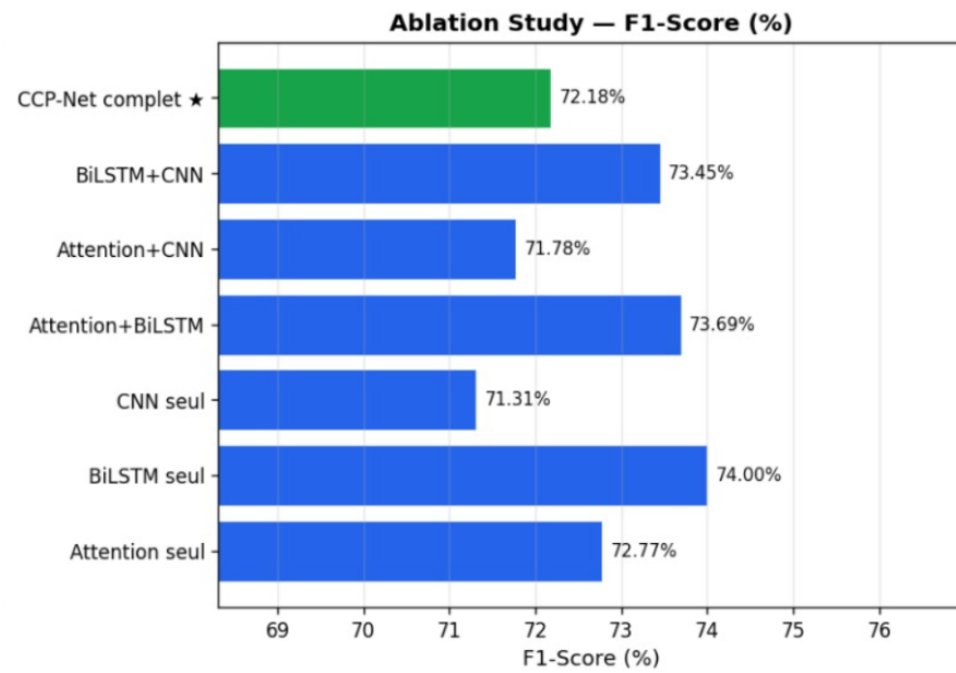


FIGURE 7.8 – Comparaison des F1-Scores de l'étude d'ablation

Conclusion

Dans ce projet, nous avons reproduit et évalué un modèle hybride de Deep Learning appelé CCP-Net, initialement proposé par Liu et al. [1], pour la prédiction de l'attrition client. Le modèle combine efficacement les avantages du Multi-Head Self-Attention [3], du BiLSTM et du CNN afin d'améliorer l'extraction des caractéristiques globales et locales.

Les résultats obtenus montrent que cette architecture hybride offre de meilleures performances que plusieurs approches classiques de Machine Learning et de Deep Learning. De plus, l'utilisation de la technique ADASYN [4] permet de réduire l'impact du déséquilibre des classes et d'améliorer la robustesse du modèle.

Comme perspectives futures, il serait intéressant d'explorer d'autres mécanismes d'attention avancés ainsi que des architectures Transformer plus complexes afin d'améliorer davantage les performances du modèle.

Bibliographie

- [1] X. Liu, G. Xia, X. Zhang, W. Ma, C. Yu, *Customer churn prediction model based on hybrid neural networks*, Scientific Reports, vol. 14, article 30707, 2024. <https://doi.org/10.1038/s41598-024-79603-9>
- [2] A. M. Khattak et al., *Customer churn prediction using composite deep learning technique*, Scientific Reports, vol. 13, article 17294, 2023. <https://doi.org/10.1038/s41598-023-44396-w>
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, *Attention is all you need*, Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- [4] H. He, Y. Bai, E. A. Garcia, S. Li, *ADASYN : Adaptive Synthetic Sampling Approach for Imbalanced Learning*, in Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN 2008), Hong Kong, pp. 1322–1328, 2008.
- [5] A. K. Ahmad, A. Jafar, K. Aljoumaa, *Customer churn prediction in telecom using machine learning in big data platform*, Journal of Big Data, vol. 6, article 28, 2019.
- [6] P. Lalwani, M. K. Mishra, J. S. Chadha, P. Sethi, *Customer churn prediction system : a machine learning approach*, Computing, vol. 104, pp. 271–294, 2021.
- [7] A. Sikri, R. Jameel, S. M. Idrees, H. Kaur, *Enhancing customer retention in telecom industry with machine learning driven churn prediction*, Scientific Reports, vol. 14, article 13097, 2024.
- [8] J. Zhou, J. Yan, L. Yang, M. Wang, P. Xia, *Customer churn prediction model based on LSTM and CNN in music streaming*, DEStech Transactions on Engineering and Technology Research, 2019.
- [9] H. Wu, *A high-performance customer churn prediction system based on self-attention*, arXiv preprint arXiv :2206.01523, 2022.
- [10] C.-F. Tsai, Y.-H. Lu, *Customer churn prediction by hybrid neural networks*, Expert Systems with Applications, vol. 36, no. 10, pp. 12547–12553, 2009.